PLAUSIBLE

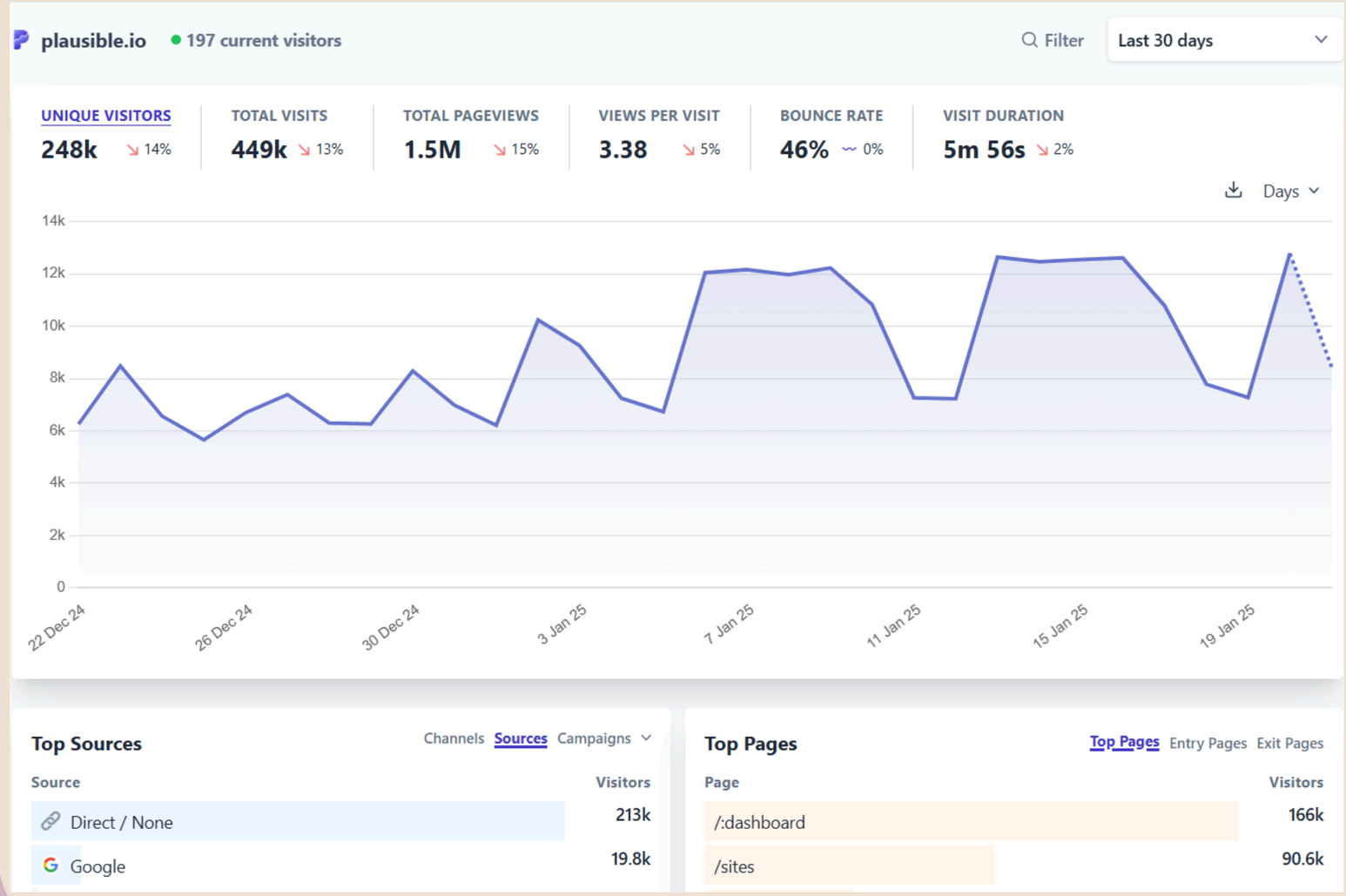
WEB ANALYTICS TOOL

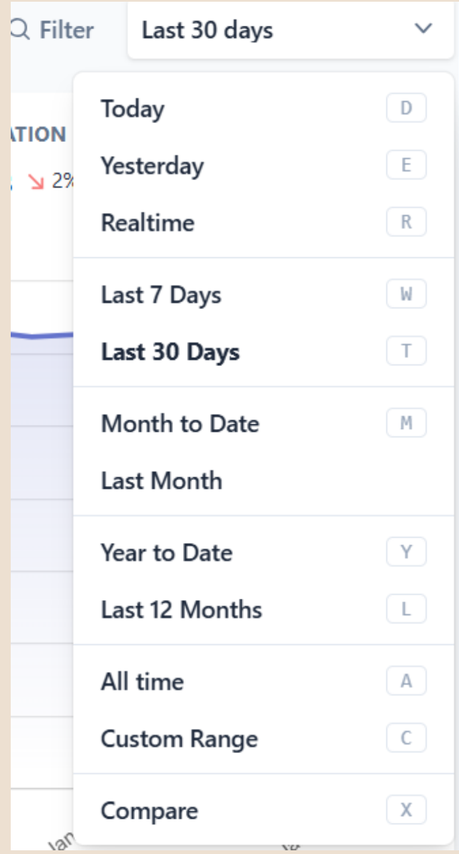
AARYA ARBAN
T11 - 05

WHAT IS PLAUSIBLE?

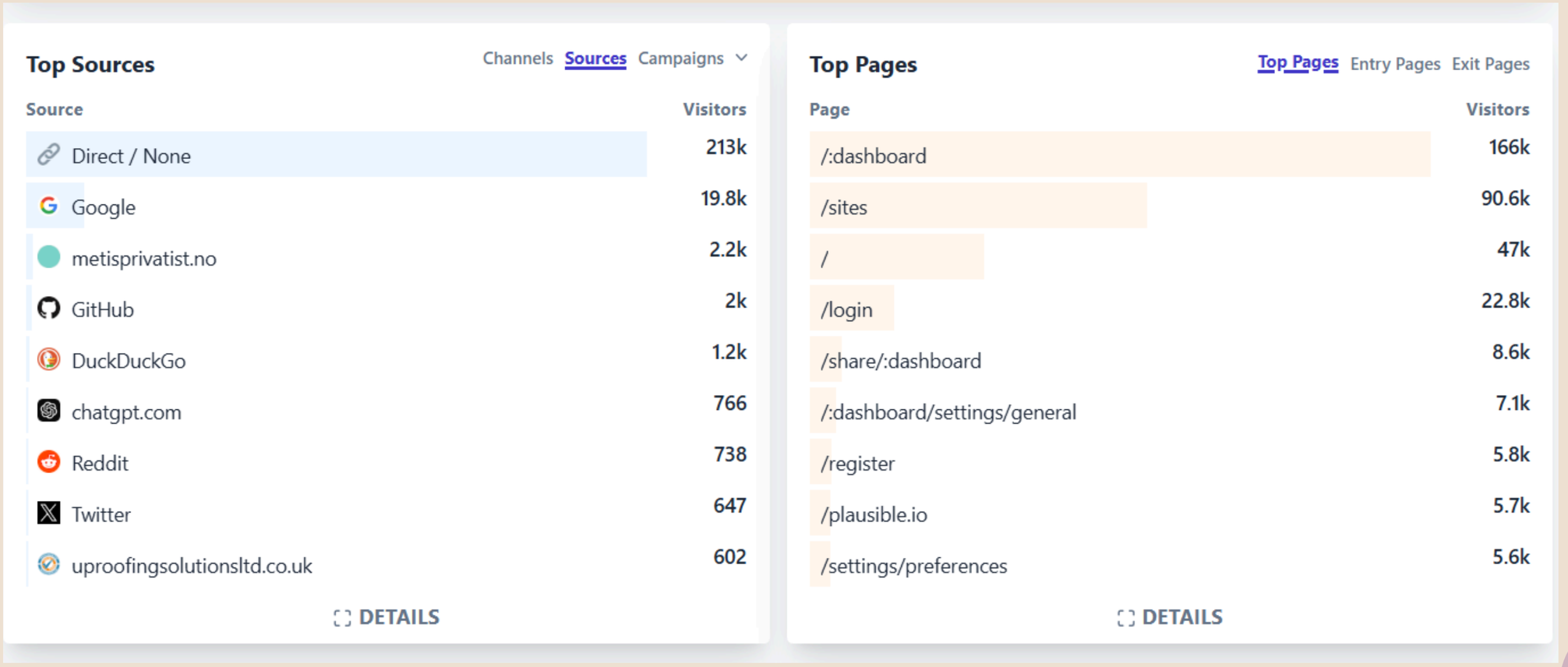
- Privacy-focused, avoids cookies and personal data tracking.
- Fully compliant with GDPR, CCPA, and PECR.
- Lightweight script for fast website performance.
- Clean dashboard with essential metrics like page views and bounce rates.
- Open-source for transparency and customization.
- Supports event tracking for user action insights.
- Subscription-based, no data monetization.
- Ideal for privacy-conscious website owners.

INTERFACE


plausible.io
197 current visitors
Filter
Last 30 days
UNIQUE VISITORS
248k ↘ 14%
TOTAL VISITS
449k ↘ 13%
TOTAL PAGEVIEWS
1.5M ↘ 15%
VIEWS PER VISIT
3.38 ↘ 5%
BOUNCE RATE
46% ~ 0%
VISIT DURATION
5m 56s ↘ 2%
Days
14k
12k
10k
8k
6k
4k
2k
0
22 Dec 24
26 Dec 24
30 Dec 24
3 Jan 25
7 Jan 25
11 Jan 25
15 Jan 25
19 Jan 25
Top Sources
Channels Sources Campaigns
Source
Visitors
Direct / None
213k
Google
19.8k
Top Pages
Top Pages Entry Pages Exit Pages
Page
Visitors
/:dashboard
166k
/sites
90.6k


Filter
Last 30 days
Today D
Yesterday E
Realtime R
Last 7 Days W
Last 30 Days T
Month to Date M
Last Month
Year to Date Y
Last 12 Months L
All time A
Custom Range C
Compare X

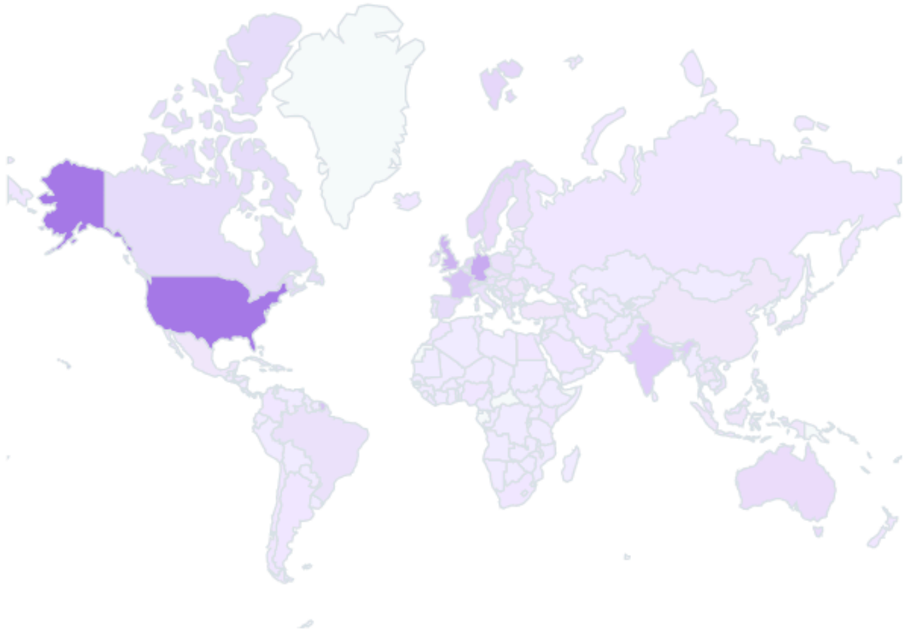
INTERFACE


Top Sources
Channels Sources Campaigns
Source
Visitors
Direct / None 213k
Google 19.8k
metisprivatist.no 2.2k
GitHub 2k
DuckDuckGo 1.2k
chatgpt.com 766
Reddit 738
Twitter 647
uproofingsolutionsltd.co.uk 602
DETAILS
Top Pages
Top Pages Entry Pages Exit Pages
Page
Visitors
/:dashboard 166k
/sites 90.6k
/ 47k
/login 22.8k
/share/:dashboard 8.6k
/:dashboard/settings/general 7.1k
/register 5.8k
/plausible.io 5.7k
/settings/preferences 5.6k
DETAILS

INTERFACE

Locations

Map Countries Regions Cities



DETAILS

Devices

Browser OS Size

Browser	Visitors	%
Chrome	143k	57.7%
Safari	65.4k	26.3%
Firefox	25.6k	10.3%
Microsoft Edge	8.5k	3.5%
Mobile App	2k	0.8%
Opera	827	0.3%
Samsung Browser	753	0.3%
Yandex Browser	130	0.1%
(not set)	110	0%

DETAILS

COMPANIES USING PLAUSIBLE

- IT Cosmetics
- Lamborghini
- Freshly Cosmetics
- University of Chicago
- Prince William County Public Schools
- Axel Springer SE
- NYX Professional Makeup

PERKS & LIMITATIONS

- Privacy-friendly, no cookies or personal data tracking.
- Fully compliant with GDPR, CCPA, and PECR.
- Lightweight script under 1 KB for faster loading.
- Clean, simple dashboard with essential metrics

- Lacks advanced features like heatmaps or segmentation.
- No free tier; paid subscription required.
- Basic reporting may not suit large enterprises.

COMPARISON

Feature	Plausible	Matomo	Countly
Privacy-Friendliness	No cookies or personal data tracking, GDPR, CCPA, and PECR compliant	Allows tracking with full control over data, GDPR compliant	Provides strong data privacy controls, GDPR compliant
Ease of Use	Simple, clean, and user-friendly interface	More complex, can be overwhelming for beginners	Intuitive interface, but requires setup
Advanced Features	Basic metrics (page views, visitors, bounce rate)	Advanced features like heatmaps, session recordings, and A/B testing	Event tracking, push notifications, and mobile app analytics
Pricing	Subscription-based, no free tier	Free for self-hosting, paid plans for cloud services	Free self-hosting, paid plans for cloud services
Deployment Options	Cloud-hosted only	Self-hosted and cloud-hosted options	Self-hosted and cloud-hosted options

THANK YOU

Aarya Arban

T11 - 05

Assignment No. 2

AIM – Write a program in typescript to implement Simple Calculator

LO2 - Understand the basic concepts of Typescript for designing web applications

THEORY:

TypeScript is a typed superset of JavaScript developed by Microsoft. It adds static typing and object-oriented features to JavaScript, helping developers catch errors at compile time and write cleaner, scalable code.

Key Features of TypeScript:

1. Static Typing – Types are checked during development.
2. Interfaces & Classes – Supports object-oriented programming.
3. Type Inference – Automatically infers variable types.
4. Cross-platform – Compiles to plain JavaScript.

Implementing a Simple Calculator in TypeScript

- Features:
  - Perform basic operations: Add, Subtract, Multiply, Divide.
  - Take two numbers and an operator.

PROGRAM:

App.jsx import ‘./App.css’ import Calculator from ‘./components/calculator’ function App() { return (

```
  <>
    <Calculator/>
  </>
 )
```

} export default App

Calculator.jsx import React, { useState } from “react”; import “./Calculator.css”; const Calculator: React.FC = () => { const [input, setInput] = useState<string>(“”);

```
 const handleClick = (value: string) => {    setInput((prev) =>
prev + value);
 };

 const handleClear = () => {    setInput(“”);
 };

 const handleCalculate = () => {
```

```
  try {       setInput(eval(input).toString()); // Using eval (safe for simple calculations)     } catch {       setInput("Error");
  }
 };

 return (
  <div className="calculator">
   <input type="text" className="calculator-screen" value={input} readOnly />
   <div className="buttons">
    {["7", "8", "9", "/"].map((btn) => (
     <button key={btn} onClick={() => handleClick(btn)}>{btn}</button>
    ))}
    {["4", "5", "6", "*"].map((btn) => (
     <button key={btn} onClick={() => handleClick(btn)}>{btn}</button>
    ))}
    {["1", "2", "3", "-"].map((btn) => (
     <button key={btn} onClick={() => handleClick(btn)}>{btn}</button>
    ))}
    {["0", ".", "+", "="].map((btn) =>
btn === "=" ? (
       <button key={btn} onClick={handleCalculate} className="equal">
        {btn}
       </button>
      ) : (
```

```
          <button key={btn} onClick={() => handleClick(btn)}>
            {btn}
          </button>
        )
      )}
      <button onClick={handleClear} className="clear">C</button>
    </div>
  </div>
 );
};  export default
Calculator;
```

---

calculator.js > calculator

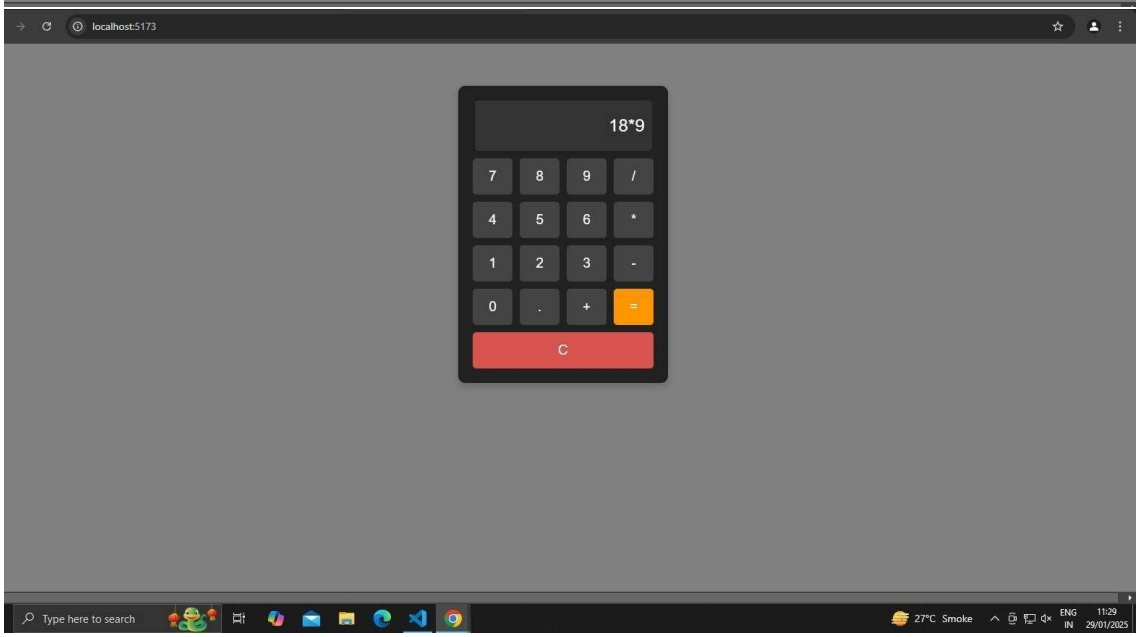
```
var _a;
var add = function (a, b) { return a + b; };
var sub = function (a, b) { return a - b; };
var mul = function (a, b) { return a * b; };
var div = function (a, b) {
    if (b === 0) {
        throw new Error("Division by zero is not allowed.");
    }
    return a / b;
};
function calculator(a, b, operation) {
    switch (operation) {
        case "add":
            return add(a, b);
        case "sub":
            return sub(a, b);
        case "mul":
            return mul(a, b);
        case "div":
            return div(a, b);
        default:
            throw new Error("Invalid operation. Supported operations are: add, sub, mul, div.");
    }
}
(_a = document.getElementById("calculate")) === null || _a === void 0 ? void 0 : _a.addEventListener("click", function () {
    var num1 = parseFloat(document.getElementById("num1").value);
    var num2 = parseFloat(document.getElementById("num2").value);
    var operation = document.getElementById("operation").value;
    var resultElement = document.getElementById("result");
```

calculator.js > calculator

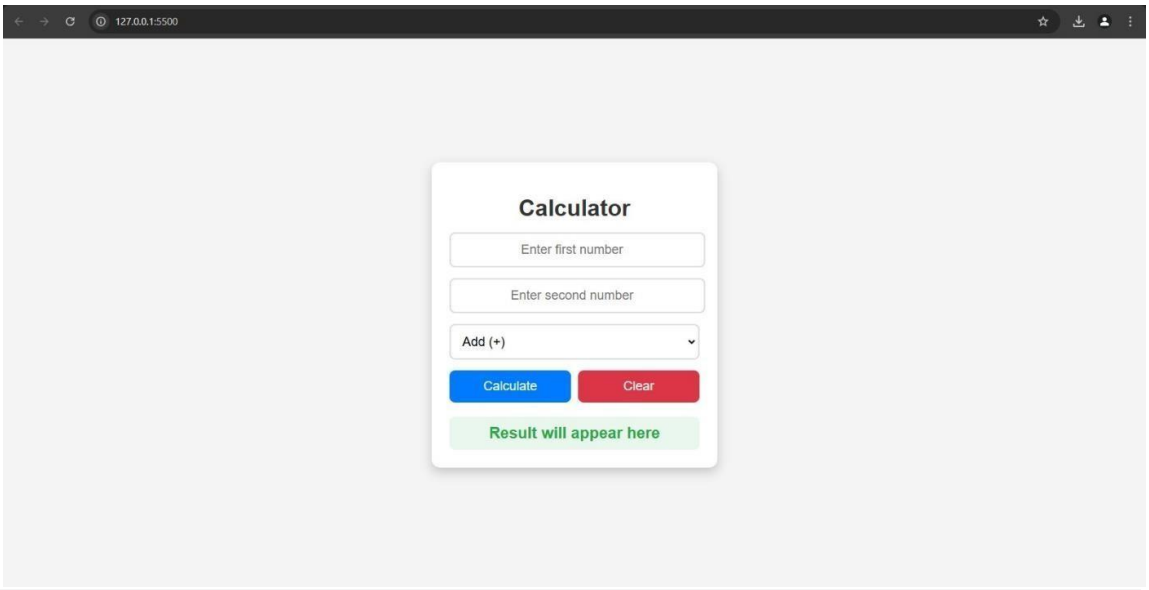
```
            throw new Error("Invalid operation. Supported operations are: add, sub, mul, div.");
    }
}
(_a = document.getElementById("calculate")) === null || _a === void 0 ? void 0 : _a.addEventListener("click", function () {
    var num1 = parseFloat(document.getElementById("num1").value);
    var num2 = parseFloat(document.getElementById("num2").value);
    var operation = document.getElementById("operation").value;
    var resultElement = document.getElementById("result");
    try {
        var result = calculator(num1, num2, operation);
        if (resultElement) {
            resultElement.textContent = "Result: ".concat(result);
        }
    }
    catch (error) {
        if (resultElement) {
            resultElement.textContent = "Error: ".concat(error.message);
        }
    }
});
```

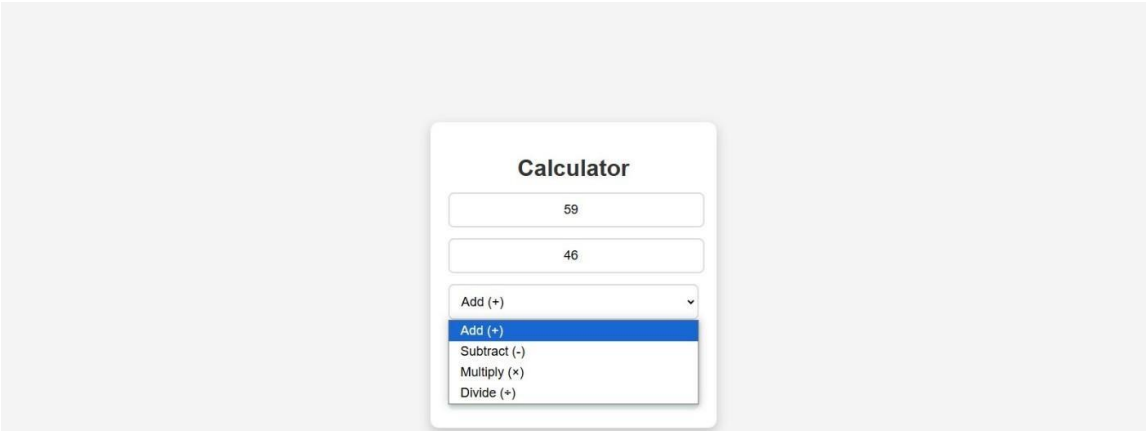
OUTPUT:

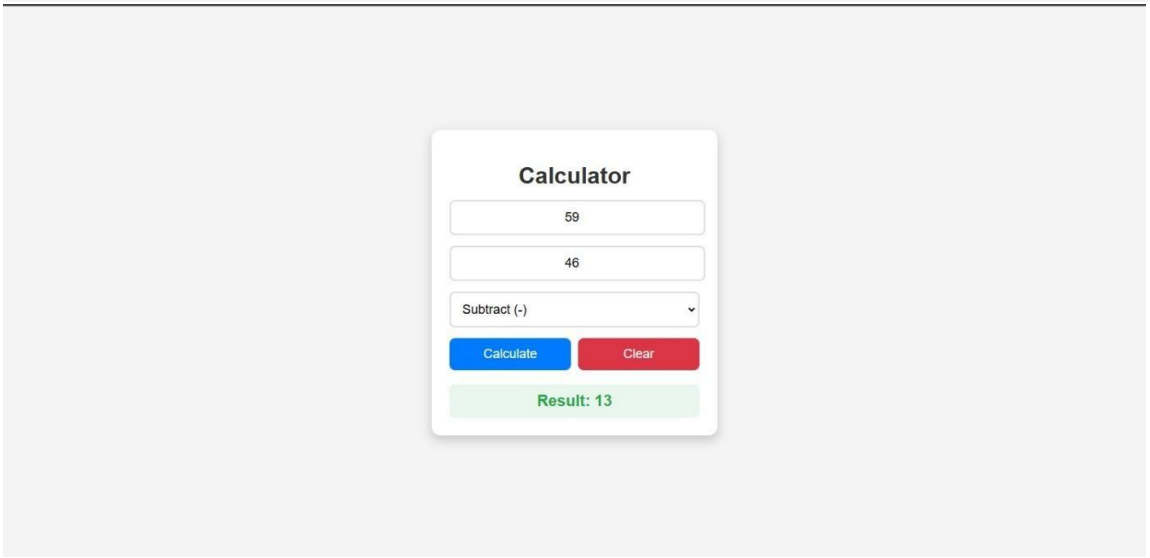

localhost:5173
162
7
8
9
/
4
5
6
*
1
2
3
-
0
.
+
=
C


localhost:5173
18*9
7
8
9
/
4
5
6
*
1
2
3
-
0
.
+
=
C
Type here to search
27°C Smoke
ENG
IN
11:29
29/01/2025


localhost:5173
7
8
9
/
4
5
6
*
1
2
3
-
0
.
+
=
C


127.0.0.1:5500
Calculator
Enter first number
Enter second number
Add (+)
Calculate
Clear
Result will appear here


Calculator
59
46
Add (+)
Add (+)
Subtract (-)
Multiply (×)
Divide (÷)


Calculator
59
46
Subtract (-)
Calculate
Clear
Result: 13

Aarya Arban

T11 – 05

Assignment No. 3

AIM – Implement the concepts of Access Modifiers, Union, Tuples using Typescript

LO2 - Understand the basic concepts of Typescript for designing web applications

THEORY:

1. Access Modifiers

Access Modifiers in TypeScript define the visibility of class members

(variables or methods). There are three main types: Modifier Description

public Accessible from anywhere (default). private Accessible only

inside the class.

protected Accessible inside the class and its subclasses.

2. Union Types

A Union allows a variable to hold more than one type. Useful when you want to accept multiple types for flexibility.

3. Tuples

A Tuple is a fixed-length array with elements of specific types. It allows storing values of different types together in a specific order.

PROGRAM:

```
// AccessModifiers.ts class
Person {     private name:
string;     protected age:
number;      public city: string;     constructor(name:
string, age: number, city: string) {        this.name = name;
this.age = age;        this.city = city;
   }
   public getDetails(): string {        return `Name: ${this.name}, Age:
${this.age}, City: ${this.city}`;
   }
}  class Employee extends Person {     private employeeId: number;
constructor(name: string, age: number, city: string, employeeId:
number) {        super(name, age, city);        this.employeeId =
employeeId;
   }
   public getEmployeeDetails(): string {        return `${this.getDetails()},
Employee ID: ${this.employeeId}`;
   }
}
// Testing access modifiers and error handling const emp = new
Employee("John Doe", 30, "New York", 12345);
try {
```

// Trying to access private member console.log(emp.name); // Error

} catch (error) { console.log("Error accessing private member 'name':", error.message);

} try

{

// Trying to access protected member console.log(emp.age); // Error

} catch (error) { console.log("Error accessing protected member 'age':", error.message);

}

console.log(emp.getEmployeeDetails()); // Accessing public method OUTPUT

\\

```
// UnionAndTuples.ts
// Union Type Example with multiple types function printIdentifier(id:
number | string | boolean): void {     if (typeof id === 'number') {
console.log(`ID (number): ${id}`);     } else if (typeof id === 'string') {
console.log(`ID (string): ${id}`);
   } else {         console.log(`ID
(boolean): ${id}`);
   }
}

// Tuple Example with multiple data types let user:
[number, string, boolean, string?]; user = [1, "Alice",
true, "Developer"]; console.log("Tuple Example:",
user);
// Testing the union type function with different
types   printIdentifier(101);   printIdentifier("XYZ-
123"); printIdentifier(true);
```

OUTPUT:


OneCompiler
PRICING
HelloWorld.ts
43dj4xugr
// UnionAndTuples.ts
// Union Type Example with multiple types
function printIdentifier(id: number | string |
if (typeof id === 'number') {
console.log(`ID (number): ${id}`);
} else if (typeof id === 'string') {
console.log(`ID (string): ${id}`);
} else {
console.log(`ID (boolean): ${id}`);
}
}
// Tuple Example with multiple data types
let user: [number, string, boolean, string?];
user = [1, "Alice", true, "Developer"];
console.log("Tuple Example:", user);
// Testing the union type function with differ
printIdentifier(101);
printIdentifier("XYZ-123");
printIdentifier(true);
STDIN
Input for the program (Optional)
Output:
Tuple Example: [1, 'Alice', true, 'Developer']
ID (number): 101
ID (string): XYZ-123
ID (boolean): true

Aarya Arban

T11 – 05

Assignment No. 4

AIM: Write a typescript program to implement inheritance for following example. Bicycle is the base class and MountainBike is derived class. Define arrow functions apply Brake() and speedUp) to decrease and increase speed of bicycle. Function toString() is to display gear and speed. Perform object initialization by using constructor.

LO2: Understand Basic Concepts of Typescript for designing web applications

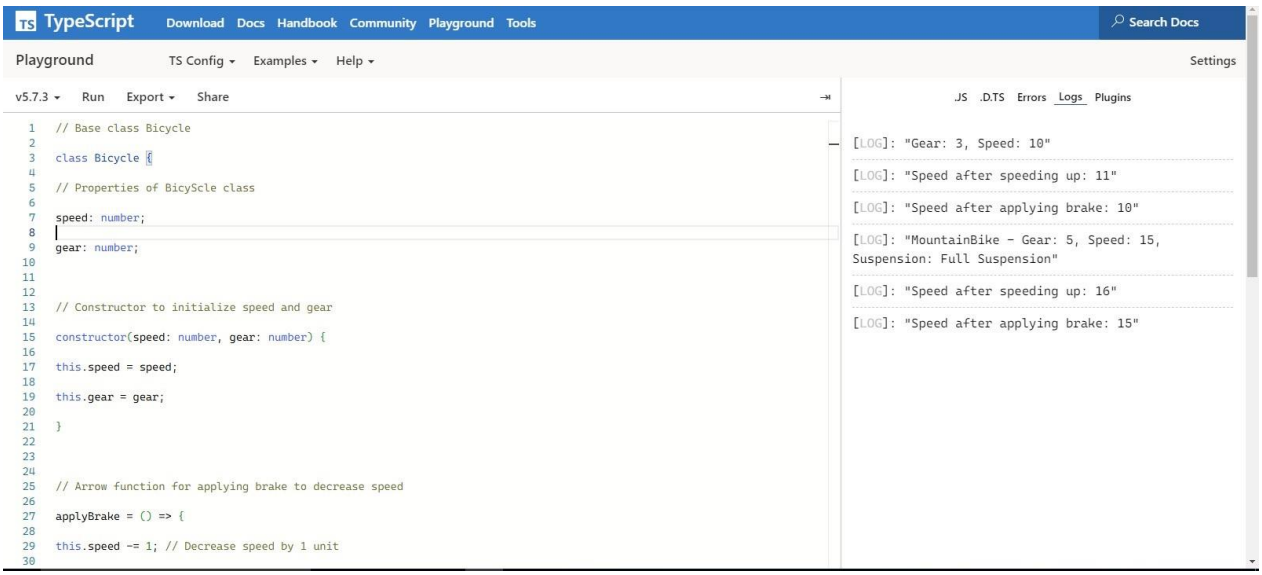
PROGRAM:

```
// Base class Bicycle class Bicycle {     gear:

number;     speed: number;

constructor(gear: number, speed:

number) {       this.gear = gear;

this.speed = speed;

   }

   // Arrow function to apply brake and decrease speed

applyBrake = (): void => {       this.speed = Math.max(0,

this.speed - 5);       console.log(`Brake applied! Current speed:

${this.speed} km/h`);
```

```
    };

    // Arrow function to increase speed     speedUp = (): void =>
{       this.speed += 5;        console.log(`Speed increased!
Current speed:
${this.speed} km/h`);
    };
    // Method to display gear and speed     toString = (): string
=> {       return `Bicycle Gear: ${this.gear}, Speed: ${this.speed}
km/h`;
    };
  }
  // Derived class MountainBike, inheriting from Bicycle   class
MountainBike extends Bicycle {     suspensionType: string;
constructor(gear: number, speed: number, suspensionType:
string) {       super(gear, speed);        this.suspensionType =
suspensionType;
    }
```

```
   // Overriding toString method to include suspensionType

toString = (): string => {

     return `MountainBike Gear: ${this.gear}, Speed: ${this.speed} km/h,
Suspension: ${this.suspensionType}`;

   };

 }

  const myBicycle = new Bicycle(3, 10);

console.log(myBicycle.toString());

myBicycle.speedUp();   myBicycle.applyBrake();


  const   myMountainBike   =   new   MountainBike(5,   15,   'Full

Suspension');                console.log(myMountainBike.toString());

myMountainBike.speedUp();    myMountainBike.applyBrake();
```

OUTPUT:


TypeScript
Download Docs Handbook Community Playground Tools
Search Docs
Playground
TS Config
Examples
Help
Settings
v5.7.3
Run
Export
Share
.JS .D.TS Errors Logs Plugins
// Base class Bicycle
class Bicycle {
// Properties of BicyScle class
speed: number;
gear: number;
// Constructor to initialize speed and gear
constructor(speed: number, gear: number) {
this.speed = speed;
this.gear = gear;
}
// Arrow function for applying brake to decrease speed
applyBrake = () => {
this.speed -= 1; // Decrease speed by 1 unit
[LOG]: "Gear: 3, Speed: 10"
[LOG]: "Speed after speeding up: 11"
[LOG]: "Speed after applying brake: 10"
[LOG]: "MountainBike - Gear: 5, Speed: 15, Suspension: Full Suspension"
[LOG]: "Speed after speeding up: 16"
[LOG]: "Speed after applying brake: 15"

Aarya Arban

T11 - 05

Assignment No. 5

AIM: Implement any 5 converters like currency converter, temperature converter, distance converter, time converter, weight converter/speed converter in Angular js

LO3 – Implement Single page applications using Angular JS

Theory:

🌐 AngularJS Overview

AngularJS is a **JavaScript-based open-source front-end web framework** mainly used to build **dynamic single-page applications (SPAs)**. It was developed by **Google** and first released in **2010**.

🔧 Key Features of AngularJS

1. **MVC Architecture** o AngularJS uses the **Model-View-Controller** pattern to separate application logic, UI, and data.
    - o This helps in organizing code and making it more manageable.
2. **Two-Way Data Binding** o Any change in the model automatically updates the view and vice versa.
    - It reduces the amount of boilerplate code needed to keep the view and model in sync.
3. **Directives** o Custom HTML attributes like ng-model, ng-bind, ng-repeat, etc., which extend the functionality of HTML. o Developers can also create **custom directives**.
4. **Dependency Injection (DI)** o AngularJS has a built-in **DI system** that helps manage and inject dependencies like services or factories efficiently.

5. **Templates** o Uses regular HTML that is enhanced with AngularJS directives to create dynamic views.
6. **Routing** o ngRoute module allows navigation between different views of the application, turning it into a **Single Page Application**.
7. **Filters** o Used to format the data displayed to the user (e.g., currency, date, uppercase/lowercase).
8. **Services & Factories** o AngularJS provides services (like $http) that handle specific tasks such as making AJAX calls.
    - You can also create your own reusable services.

PROGRAM:

INDEX.HTML

```
<!DOCTYPE html>

<html lang="en" ng-app="converterApp">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Converter App</title>

  <link rel="stylesheet" href="style.css">

  <script src="https://code.angularjs.org/1.8.2/angular.min.js"></script>

  <style>       body {

fontfamily: Arial, sans-serif;

margin: 0;          padding: 0;

background-color: #f4f4f4;
```

```
        }
    </style>
</head>
<body>
<div class="container" ng-controller="ConverterController">
    <h2>Conversion Tools</h2>
<!-- Currency Converter -->
    <div class="converter-box">
        <h3>Currency Converter: USD to INR</h3>
        <input type="number" ng-model="currencyAmount" placeholder="Enter
Amount in USD" />
        <button ng-click="convertCurrency()">Convert</button>
        <div class="result" ng-if="convertedCurrency !== undefined">
            {{currencyAmount}} USD = {{convertedCurrency}} INR
        </div>
    </div>
    <!-- Temperature Converter -->
    <div class="converter-box">
        <h3>Temperature Converter: Celsius to Fahrenheit</h3>
        <input type="number" ng-model="tempCelsius" placeholder="Enter
Temperature in Celsius" />
```

```
    <button ng-click="convertTemperature()">Convert</button>
    <div class="result" ng-if="convertedTemp !== undefined">
      {{tempCelsius}} °C = {{convertedTemp}} °F
    </div>
  </div>
  <!-- Distance Converter -->
  <div class="converter-box">
    <h3>Distance Converter: Kilometers to Miles</h3>
    <input type="number" ng-model="distanceKm" placeholder="Enter Distance in Kilometers" />
    <button ng-click="convertDistance()">Convert</button>
    <div class="result" ng-if="convertedDistance !== undefined">
      {{distanceKm}} Km = {{convertedDistance}} Miles
    </div>
  </div>
  <!-- Time Converter -->
  <div class="converter-box">
    <h3>Time Converter: Hours to Minutes</h3>
    <input type="number" ng-model="timeHours" placeholder="Enter Time in Hours" />
    <button ng-click="convertTime()">Convert</button>
```

```
      <div class="result" ng-if="convertedTime !== undefined">
        {{timeHours}} Hour(s) = {{convertedTime}} Minutes
      </div>
    </div>
    <!-- Weight Converter -->
    <div class="converter-box">
      <h3>Weight Converter: Kilograms to Pounds</h3>
      <input type="number" ng-model="weightKg" placeholder="Enter Weight in
Kilograms" />
      <button ng-click="convertWeight()">Convert</button>
      <div class="result" ng-if="convertedWeight !== undefined">
        {{weightKg}} Kg = {{convertedWeight}} lbs
      </div>
    </div>
</div> <script>    var app =
angular.module('converterApp', []);
app.controller('ConverterController',
function($scope) {
```

```
    // Currency conversion (example: 1 USD = 82
INR)        $scope.convertCurrency = function() {
if ($scope.currencyAmount) {
            $scope.convertedCurrency = $scope.currencyAmount * 87;
        }
    };

    // Temperature conversion (Celsius to Fahrenheit)
$scope.convertTemperature = function() {           if
($scope.tempCelsius) {
            $scope.convertedTemp = ($scope.tempCelsius * 9/5) + 32;
        }
    };
    // Distance conversion (Kilometers to Miles)
$scope.convertDistance = function() {           if
($scope.distanceKm)                                {
$scope.convertedDistance = $scope.distanceKm *
0.621371;
        }
    };
```

```
    // Time conversion (Hours to
Minutes)       $scope.convertTime =
function() {           if
($scope.timeHours) {
            $scope.convertedTime = $scope.timeHours * 60;
        }
    };
    // Weight conversion (Kilograms to Pounds)
$scope.convertWeight = function() {           if
($scope.weightKg) {
            $scope.convertedWeight = $scope.weightKg * 2.20462;
        } };
  });
</script>
</body>
</html>
```

STYLE.CSS

```
.container {     width: 70%;     margin: 50px
auto;     background-color: #fff;     padding:
30px;     box-shadow: 0 4px 8px rgba(0, 0,
0, 0.1);
} h2 {     text-align:
center; }
.converterbox {
margin:
20px 0; }
.converter-box input,
.converter-box select {
width: 100%;     padding:
10px;     margin: 10px 0;
border: 1px solid #ccc;
border-radius: 4px;
}
.converter-box button {
padding: 10px 20px;
```

```
background-color: rgb(110,
196, 70);     color: white;
border: none;     border-radius:
4px;     cursor: pointer;
}
.converter-box button:hover {
backgroundcolor: #0056b3;  } .result {
textalign: center;
font-size: 1.2em;
color: #333;
}
```

OUTPUT:

Conversion Tools

Currency Converter: USD to INR

1

Convert

1 USD = 87 INR

Temperature Converter: Celsius to Fahrenheit

100

Convert

100 °C = 212 °F

Distance Converter: Kilometers to Miles

1000

Convert

1000 Km = 621.371 Miles

Time Converter: Hours to Minutes

1

Convert

1 Hour(s) = 60 Minutes

Weight Converter: Kilograms to Pounds

10

Convert

10 Kg = 22.0462 lbs

Aarya Arban

T11 - 05

Assignment No. 6

AIM - Implement student registration form with validation using angularjs.

LO3 – Implement Single page application using AngularJS

Theory:

A **Single Page Application (SPA)** is a web application that loads a single HTML page and dynamically updates the content as the user interacts with the app, without reloading the entire page. **AngularJS** is well-suited for building SPAs due to its powerful features like **two-way data binding**, **routing**, and **dependency injection**. Using the ngRoute module or ui-router, developers can define different views (or templates) and link them to specific controllers, enabling dynamic navigation between pages like home, about, and contact—while staying on the same HTML file.

To implement a SPA in AngularJS, the developer first defines routes using the $routeProvider service inside the app's configuration block. Each route points to a template and a controller, which handles the logic for that particular view. When the user clicks a link or performs an action that triggers a route change, AngularJS dynamically loads the relevant view without a full-page reload. This provides a faster, smoother user experience similar to desktop applications. SPAs built with AngularJS are ideal for creating interactive and responsive web apps such as dashboards, chat apps, or e-commerce platforms.

PROGRAM:

index.html

```
<!DOCTYPE html>
```

```
<html lang="en" ng-app="studentApp">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Registration Form</title>
    <!-- Bootstrap CSS -->
    <link
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css "
rel="stylesheet">
    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></scrip
t>
    <link rel="stylesheet" href="style.css">
</head>
<body ng-controller="StudentController">
    <div class="container mt-5">
        <h2 class="text-center">Student Registration Form</h2>
        <!-- Success Message -->
        <div class="alert alert-success" ng-if="formSubmitted">
            <strong>Success!</strong> Your form has been submitted successfully.
            <pre>{{ student | json }}</pre>
        </div>
        <!-- Form -->
```

```
<form name="registrationForm" ng-submit="submitForm()" novalidate>

    <!-- Full Name -->

    <div class="form-group">

      <label for="fullName">Full Name:</label>

      <input      type="text"  id="fullName"        name="fullName"
    class="formcontrol" ng-model="student.fullName" ng-pattern="/^[A-Za-
z\s]+$/" required>

      <small class="form-text text-danger"
ngshow="registrationForm.fullName.$touched &&
registrationForm.fullName.$invalid">Full Name is required and must contain only
letters.</small>

    </div>

    <!-- Email -->

    <div class="form-group">

      <label for="email">Email:</label>

      <input type="email" id="email" name="email" class="form-control"
ngmodel="student.email" required>

      <small class="form-text text-danger"
ngshow="registrationForm.email.$touched &&
registrationForm.email.$invalid">Valid email is required.</small>

    </div>

    <!-- Phone Number -->

    <div class="form-group">

      <label for="phone">Phone Number:</label>
```

```
        <input type="text" id="phone" name="phone" class="form-control" ng-model="student.phone" ng-pattern="/^[0-9]{10}$/" required>

        <small class="form-text text-danger" ngshow="registrationForm.phone.$touched && registrationForm.phone.$invalid">Phone number must be 10 digits.</small>

    </div>

    <!-- Date of Birth -->

    <div class="form-group">

        <label for="dob">Date of Birth:</label>

        <input type="date" id="dob" name="dob" class="form-control" ng-model="student.dob" ng-change="calculateAge()" required>

        <small class="form-text text-danger" ngshow="registrationForm.dob.$touched && registrationForm.dob.$invalid">Date of Birth is required.</small>

        <small class="form-text text-danger" ngshow="!registrationForm.dob.$invalid && student.age < 18">Student must be at least 18 years old.</small>

    </div>

    <!-- Gender -->

    <div class="form-group">

        <label>Gender:</label><br>

        <input type="radio" ng-model="student.gender" value="Male" required> Male

        <input type="radio" ng-model="student.gender" value="Female" required> Female
```

```
            <small class="form-text text-danger"
ngshow="registrationForm.gender.$touched &&
registrationForm.gender.$invalid">Gender is required.</small>

        </div>

        <!-- Password -->

        <div class="form-group">

            <label for="password">Password:</label>

            <input type="password" id="password" name="password"
class="formcontrol" ng-model="student.password" required>

            <small class="form-text text-danger"
ngshow="registrationForm.password.$touched &&
registrationForm.password.$invalid">Password is required.</small>

        </div>

        <!-- Confirm Password -->

        <div class="form-group">

            <label for="confirmPassword">Confirm Password:</label>

            <input type="password" id="confirmPassword" name="confirmPassword"
class="form-control" ng-model="student.confirmPassword"
ngrequired="student.password" ng-pattern="student.password" required>

            <small class="form-text text-danger" ng-
show="registrationForm.confirmPassword.$touched &&
registrationForm.confirmPassword.$invalid">Passwords must match.</small>

        </div>

        <!-- Submit Button -->
```

```
        <button type="submit" class="btn btn-primary"
ngdisabled="registrationForm.$invalid">Register</button>

      </form>

  </div>

  <!-- Bootstrap JS & jQuery (Needed for Bootstrap 4's JS components) -->

  <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>

  <script
src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.5.2/dist/umd/popper.min.j
s"></script>

  <script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></
script>

  <script src="app.js"></script>

</body>

</html>
```

app.js

```
// app.js var app = angular.module('studentApp', []);


app.controller('StudentController', ['$scope', function ($scope) {

  // Initialize the student object

  $scope.student = {};

  $scope.formSubmitted = false;

  $scope.student.age = null;
```

```
    // Function to calculate age based on the Date of Birth
    $scope.calculateAge = function () {        if
($scope.student.dob) {            var dob = new
Date($scope.student.dob);            var today = new Date();            var
age = today.getFullYear() - dob.getFullYear();            var m =
today.getMonth() - dob.getMonth();

            if (m < 0 || (m === 0 && today.getDate() < dob.getDate())) {                age--;
            }
            $scope.student.age = age;
        }
    };

    // Function to handle form submission
$scope.submitForm = function () {        if
($scope.registrationForm.$valid) {
            $scope.formSubmitted = true;
        }
    };
}]);
```

style.css body { font-family:

Arial, sans-serif; padding:

20px;

} label { display: block;

margin-bottom: 5px;

}

input { margin-bottom:

10px; padding: 8px;

width: 100%; max-width:

300px;

}

button { padding: 10px 15px;

background-color: #4CAF50; color:

white; border: none; cursor:

pointer;

}

button:disabled { backgroundcolor:

#ccc;

}

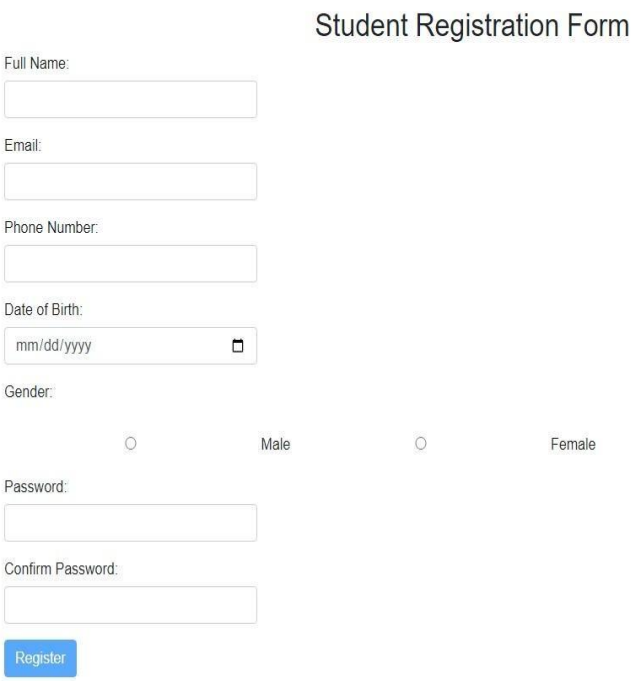
span { color: red; font-size:

12px;

}

```
.form-container {     maxwidth:
400px;     margin: 0 auto;

}
h2 {     text-align:
center;
}
.error {     color: red;     font-
size: 12px;
}
```

OUTPUT:



Student Registration Form

Full Name:

Full Name is required and must contain only letters.

Email:

Valid email is required.

Phone Number:

Phone number must be 10 digits.

Date of Birth:

mm/dd/yyyy

Date of Birth is required.

Gender:

Male Female

Password:

Password is required.

Confirm Password:

Aarya Arban

T11 - 05

Aim – Implement Single page application using ng-route.

LO3 - Implement Single page application using AngularJS

Theory:

To implement a **Single Page Application (SPA)** using AngularJS, the ngRoute module plays a key role in enabling client-side routing. This module allows developers to define **routes** that map URLs to specific templates and controllers. Instead of navigating to a new HTML page, AngularJS dynamically loads different views into a single page using the <ng-view> directive. This makes the application faster and more responsive, as only parts of the page are updated when a user navigates, without triggering a full page reload.

In practice, you first include the angular-route.js library and configure routing using the $routeProvider service. Each route is linked to a specific template and controller. For example, a route like /home may load home.html and be controlled by HomeController, while /about loads about.html with AboutController. The <ng-view> directive placed in the main HTML acts as a placeholder where the matched views are rendered. This structure makes it easy to build modular, maintainable SPAs where navigation between different sections feels seamless and instantaneous.

Program:

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>
```

```
    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initialscale=1.0">

    <title>AngularJS with ngRoute</title>

    <link rel="stylesheet" href="style.css">

    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min
.js"></script>

    <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angularroute.mi
n.js">< /script>

</head>

<body ng-app="myApp">



    <div class="navbar">

        <ul>

            <li><a href="#!/">Home</a></li>

            <li><a href="#!/about">About</a></li>

            <li><a href="#!/services">Services</a></li>

            <li><a href="#!/contact">Contact</a></li>

        </ul>

    </div>
```

```
    <div ng-view></div>

    <script src="app.js"></script>
</body>
</html>
```

Style.css

```
* {     margin: 0;     padding:
0;     box-sizing: border-box;
}

body {     font-family: 'Arial', sansserif;
                            line-height:  1.6;
background-color: #f4f4f4;     color:
#333;
}

.navbar {     background-color:
#333;     padding: 15px;
textalign: center;
}
```

```
.navbar ul {     list-
styletype: none;
padding: 0;     margin: 0;
}

.navbar ul li {     display:
inline;     margin-right:
20px;
}

.navbar ul li a {     color: white;
text-decoration: none;
fontweight: bold;     text-
transform: uppercase;
}

.navbar ul li a:hover {     color:
#4CAF50;
}
```

```
.navbar ul li a.active {     color:
#4CAF50;     font-weight: bold;
}

/* Hero Section */
.hero {     position:
relative;     width:
100%;     height: 60vh;
}

.hero-image {     width:
100%;     height:
100%;     object-fit:
cover;     opacity: 0.7;
}

.hero-content {     position: absolute;     top: 50%;     left: 50%;     transform:
translate(-50%, -50%);     text-align: center;     color: white;     max-width:
90%;  }

.hero h2 {     font-size:
```

```
2.5rem;     margin-bottom:
10px;
}

.hero p {     font-size:
1.2rem;
}

/* Feature Section */
.featuresection {     text-align:
center;     margin-top: 40px;
}

.feature-section h3 {     font-size:
2rem;     margin-bottom: 10px;
}

.feature-video {     text-align:
center;     margin-top: 20px;
}
```

```
.feature-video video {     width:
100%;     max-width: 800px;
border-radius: 8px;
}

/* About Section */ .about-section {
display: flex;     align-items: center;
justify-content: space-between;     margin-
top: 40px;
}

.about-image {     width:
50%;     height: auto;
border-radius: 8px;
}

.about-content {     width:
45%;
}

.about-content h2 {     font-size:
```

```
2rem;     margin-bottom: 10px;
}

.about-content ul {     list-
style-type:               none;
padding-left: 20px;
}

.about-content li {     font-size:
1.1rem;     margin-bottom:
10px;
}

.about-video {     margin-top:
40px;     text-align: center;
}

.about-video video {     width:
100%;     max-width: 800px;
border-radius: 8px;
}
```

```
/* Services Section */
.servicessection {     margin-top:
40px;     text-align: center;
}

.services-cards {     display: flex;     justify-
content: space-around;     margin-top:
30px;
}

.service-card {       background-color: #fff;
padding:  20px;          border-radius:  8px;
boxshadow: 0 0 10px rgba(0, 0, 0, 0.1);
width:
30%;     text-align: center;
}

.service-card img {
width: 100%;     height:
auto;     border-radius:
```

```
8px;
}

.service-card h3 {     margin-top:
15px;     font-size: 1.5rem;
}

.service-card p {     font-size:
1.1rem;     margin-top: 10px;
}

.services-video {     text-align:
center;     margin-top: 40px;
}

.services-video video {
width: 100%;     maxwidth:
800px;     borderradius: 8px;
}
```

```
/* Contact Section */
.contactsection {     margin-top:
40px;     text-align: center;
}

.contact-details h3 {     font-size:
1.8rem;     margin-top: 20px;
}

.contact-details p {     font-size: 1.1rem;     margin: 10px 0;
}

.contact-video {     text-align:
center;     margin-top: 40px;
}

.contact-video video {     width:
100%;     max-width: 800px;
border-radius: 8px;
}
```

```
/* Footer */ footer {

text-align: center;

margin-top: 30px;

color: #777;

fontsize: 0.9rem;

}

app.js angular.module('myApp',

['ngRoute'])

.config(function($routeProvider) {

      $routeProvider

         .when('/', {

template: `

               <div class="hero">

                  <img src="img1.jpg" alt="Home Image" class="heroimage"/>

                  <div class="hero-content">

                     <h2>Welcome to the Home Page!</h2>

                     <p>We are excited to have you here. This is your one-stop destination for all the information you need about our services and company. Please feel free to explore!</p>

                     <p>Our mission is to deliver quality and innovative solutions for our customers.</p>
```

```
            </div>
        </div>
        <div class="feature-section">
            <h3>Why Choose Us?</h3>
            <p>Our mission is to deliver quality and innovative solutions for our customers. We specialize in web development, mobile app development, and digital marketing.</p>
            <div class="feature-video">
                <video controls>
                    <source src="video1.MP4">
                    Your browser does not support the video tag.
                </video>
            </div>
        </div>
    `
})
.when('/about', {
template: `
        <div class="about-section">
            <img src="img1.jpg" alt="About Us" class="aboutimage"/>
            <div class="about-content">
```

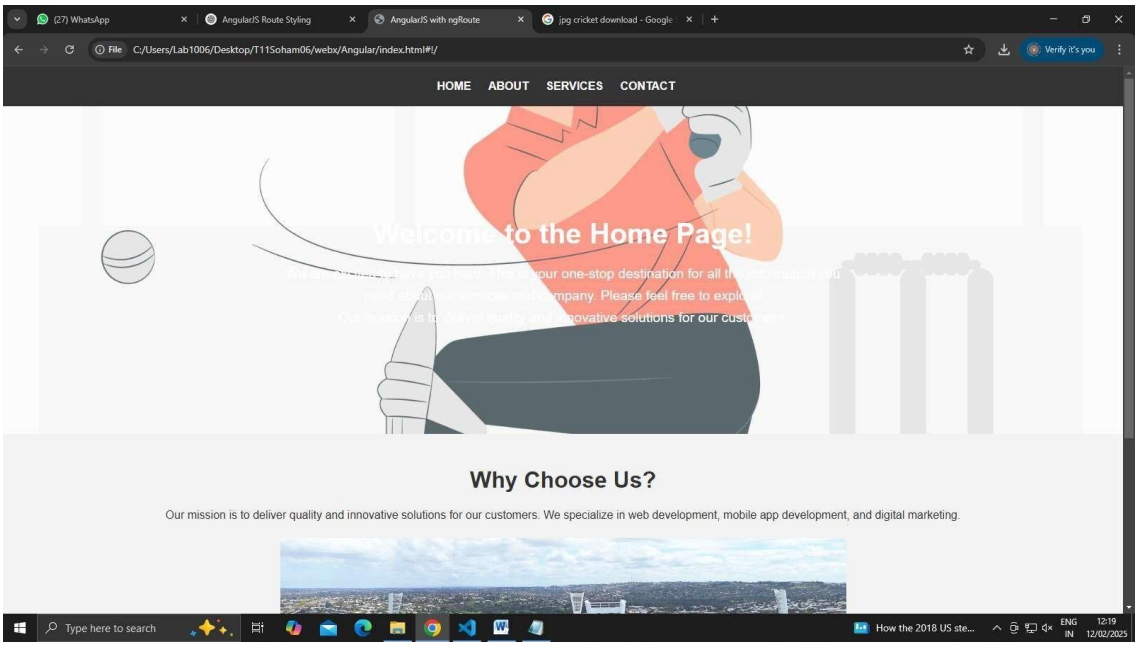
```
                <h2>About Us</h2>

                <p>We are a passionate team dedicated to providing exceptional digital solutions. We specialize in creating high-quality websites, mobile apps, and more.</p>

                <p>Our mission is to empower businesses to succeed in a digital world.</p>

                <h3>Our Values:</h3>

                <ul>

                    <li>Innovation</li>

                    <li>Integrity</li>

                    <li>Customer-Centric</li>

                </ul>

            </div>

        </div>

        <div class="about-video">

            <video controls>

                <source src="video1.MP4" >

                Your browser does not support the video tag.

            </video>

        </div>

    `

})
```

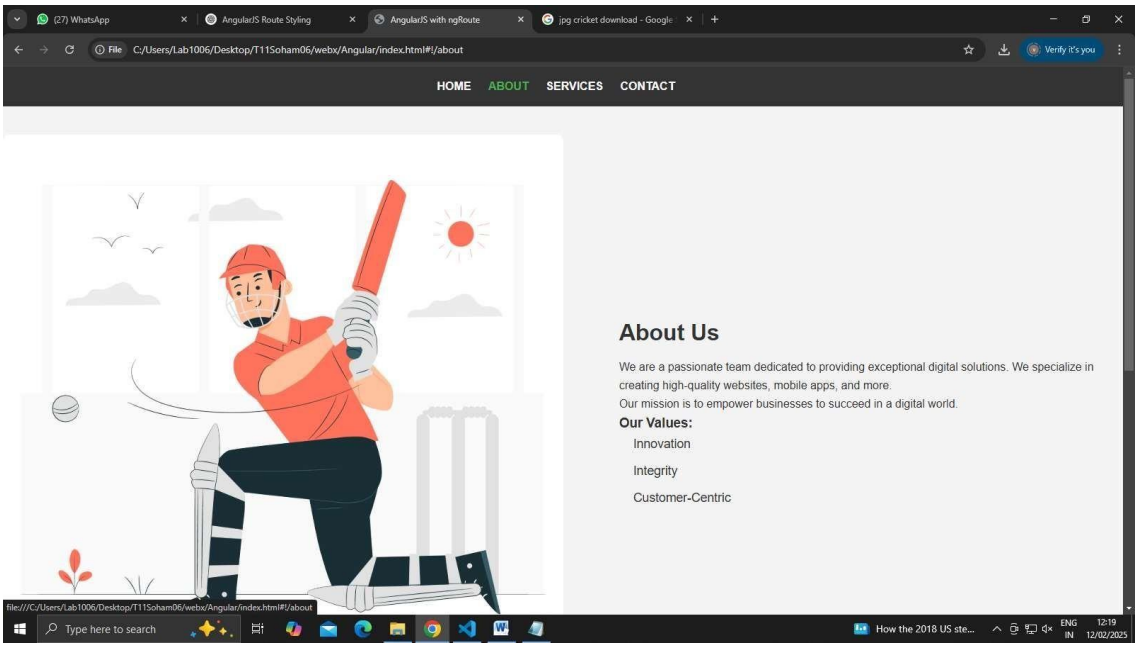
```
.when('/services', {

template: `

<div class="services-section">

<h2>Our Services</h2>

<p>Explore the wide range of services we offer to help your business grow. Whether you need a website, mobile app, or marketing services, we have you covered.</p>

<div class="services-cards">

<div class="service-card">

<img src="img1.jpg" alt="Web Development"/>

<h3>Web Development</h3>

<p>Crafting beautiful and responsive websites</p>

</div>

<div class="service-card">

<img src="img1.jpg" alt="App Development"/>

<h3>Mobile App Development</h3>

<p>Creating innovative apps for iOS and Android</p>

</div>

<div class="service-card">

<img src="img1.jpg" alt="Digital Marketing"/>
```

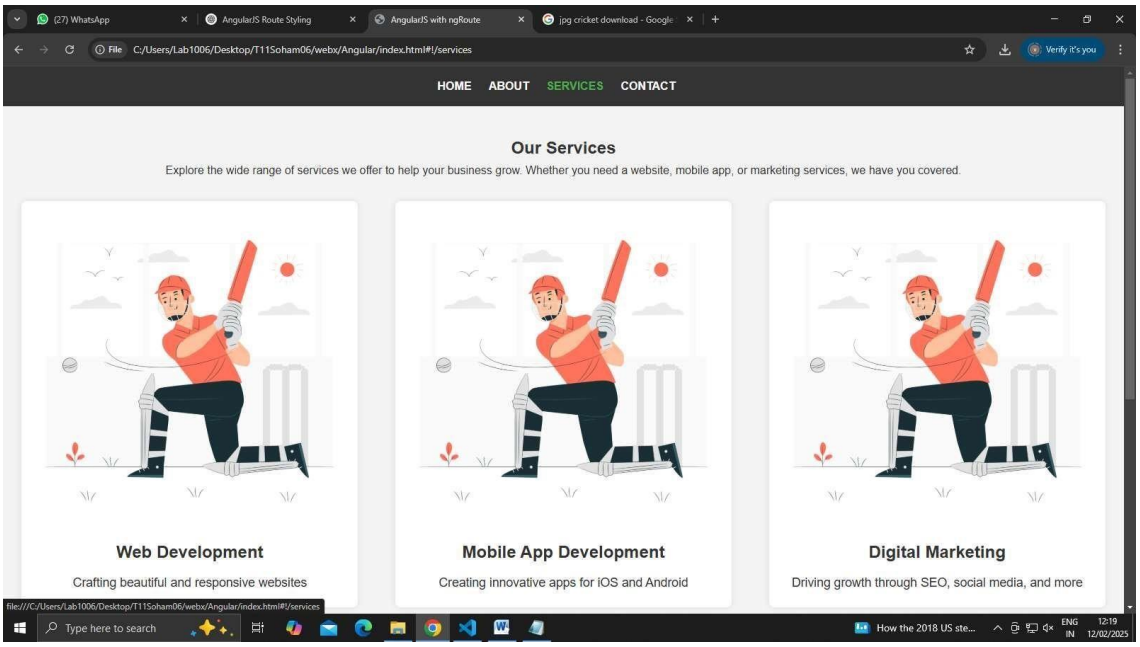
```
                    <h3>Digital Marketing</h3>
                    <p>Driving growth through SEO, social media, and more</p>
                </div>
            </div>
            <div class="services-video">
                <video controls>
                    <source src="video1.MP4" >
                    Your browser does not support the video tag.
                </video>
            </div>
        </div>
    `
    })
    .when('/contact', {
template: `
        <div class="contact-section">
            <h2>Contact Us</h2>
            <p>We'd love to hear from you! Whether you have a question, feedback, or need assistance, we're here to help.</p>
<div class="contact-details">
                <h3>Get in Touch</h3>
```

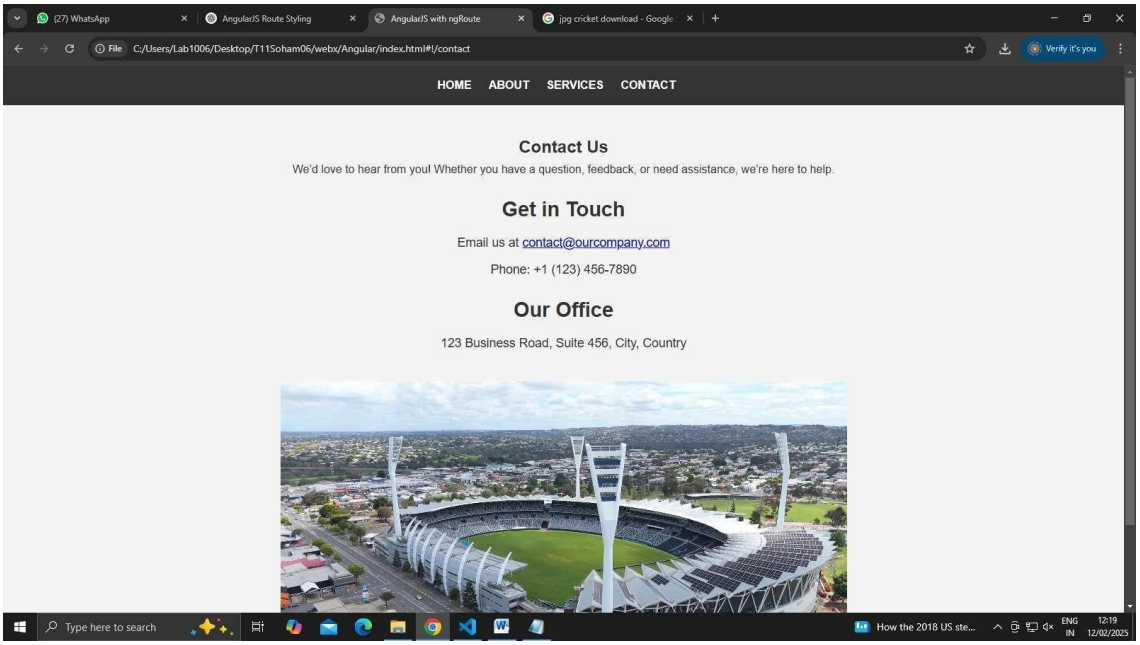
```
                    <p>Email us at <a
href="mailto:contact@ourcompany.com">contact@ourcompany.com</
a></p>

                    <p>Phone: +1 (123) 456-7890</p>

                    <h3>Our Office</h3>

                    <p>123 Business Road, Suite 456, City, Country</p>

                </div>

                <div class="contact-video">

                    <video controls>

                        <source src="video1.MP4" >

                        Your browser does not support the video tag.

                    </video>

                </div>

            </div>

        `

    })

    .otherwise({

redirectTo: '/'

    });

});
```

OUTPUT:


HOME ABOUT SERVICES CONTACT
Welcome to the Home Page!
Why Choose Us?
Our mission is to deliver quality and innovative solutions for our customers. We specialize in web development, mobile app development, and digital marketing.


HOME ABOUT SERVICES CONTACT
About Us
We are a passionate team dedicated to providing exceptional digital solutions. We specialize in creating high-quality websites, mobile apps, and more.
Our mission is to empower businesses to succeed in a digital world.
Our Values:
Innovation
Integrity
Customer-Centric


(27) WhatsApp
AngularJS Route Styling
AngularJS with ngRoute
jpg cricket download - Google
File C:/Users/Lab1006/Desktop/T11Soham06/webx/Angular/index.html#!/services
Verify it's you
HOME ABOUT SERVICES CONTACT
Our Services
Explore the wide range of services we offer to help your business grow. Whether you need a website, mobile app, or marketing services, we have you covered.
Web Development
Crafting beautiful and responsive websites
Mobile App Development
Creating innovative apps for iOS and Android
Digital Marketing
Driving growth through SEO, social media, and more
file:///C:/Users/Lab1006/Desktop/T11Soham06/webx/Angular/index.html#!/services
Type here to search
How the 2018 US ste...
ENG IN
12:19 12/02/2025


(27) WhatsApp
AngularJS Route Styling
AngularJS with ngRoute
jpg cricket download - Google
File C:/Users/Lab1006/Desktop/T11Soham06/webx/Angular/index.html#!/contact
Verify it's you
HOME ABOUT SERVICES CONTACT
Contact Us
We'd love to hear from you! Whether you have a question, feedback, or need assistance, we're here to help.
Get in Touch
Email us at contact@ourcompany.com
Phone: +1 (123) 456-7890
Our Office
123 Business Road, Suite 456, City, Country
Type here to search
How the 2018 US ste...
ENG IN
12:19 12/02/2025

Aarya Arban

T11 – 05

Assignment No. 8

AIM: To implement five built-in helper functions in AngularJS for various tasks such as data manipulation, DOM manipulation, and more

LO3 – Implement Single page application using AngularJS

THEORY:

AngularJS provides several built-in helper functions to simplify common tasks in web development. These functions are designed to enhance productivity and maintainability by offering pre-built solutions for recurring problems. By leveraging these helper functions, developers can streamline their code and focus on building robust applications more efficiently.

CODE:

```
<!DOCTYPE html>
<html ng-app="helperFunctionsApp"> <head>
    <title>AngularJS Helper Functions</title>
    <script
    src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.
js"></script>
</head>
<body>


<div ng-controller="HelperFunctionsController">
    <h2>Data Manipulation</h2>
    <p>Original Data: {{originalData}}</p>
    <p>Copied Data: {{copiedData}}</p>


    <h2>DOM Manipulation</h2>
                <button ng-click="toggleVisibility()">Toggle Visibility</button>
    <p ng-show="isVisible">This paragraph is visible.</p>
```

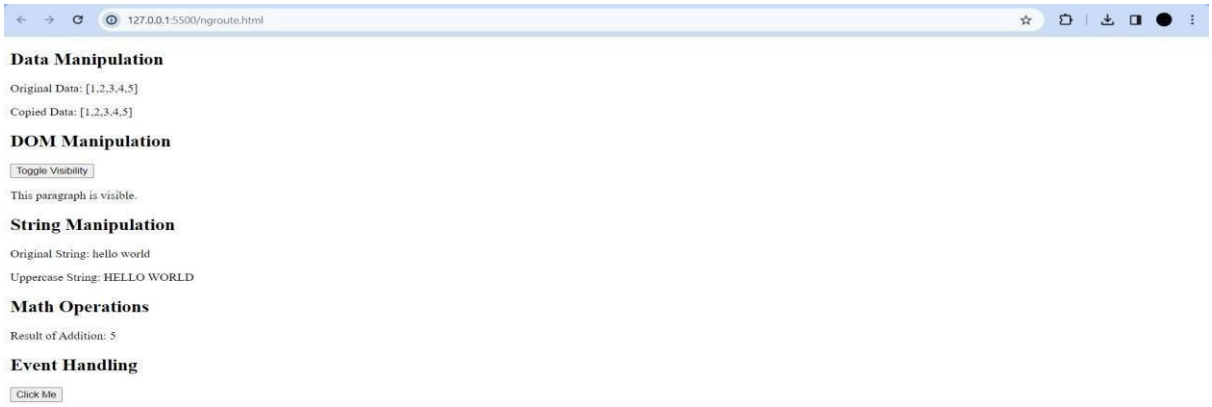
```
    <h2>String Manipulation</h2>
    <p>Original String: {{originalString}}</p>
    <p>Uppercase String: {{uppercasedString}}</p>

    <h2>Math Operations</h2>
<p>Result of Addition: {{additionResult}}</p>
```

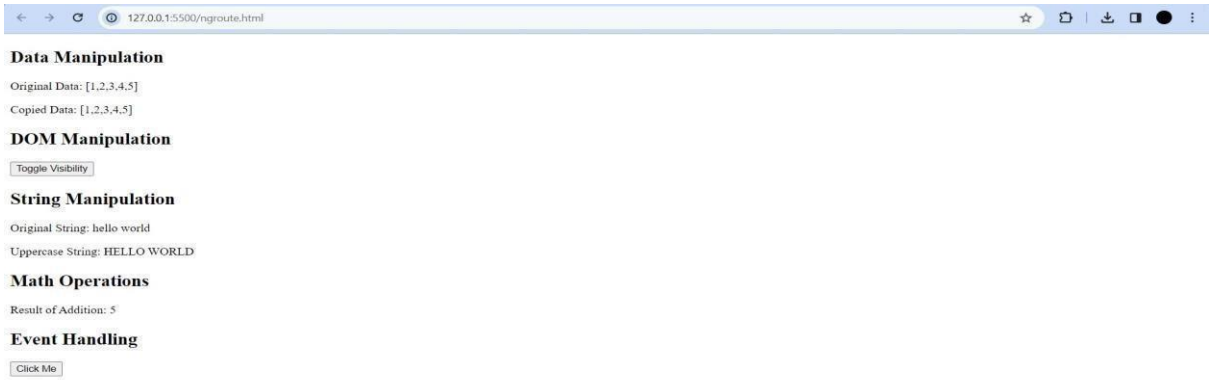
```
    <h2>Event Handling</h2>
    <button ng-click="handleClick()">Click Me</button>
    <p ng-show="isClicked">Button Clicked!</p>  </div>

<script>  angular.module('helperFunctionsApp', [])
        .controller('HelperFunctionsController', function ($scope) {
            // Data Manipulation
            $scope.originalData = [1, 2, 3, 4, 5];
            $scope.copiedData = angular.copy($scope.originalData);

            // DOM Manipulation
            $scope.isVisible = true;
```

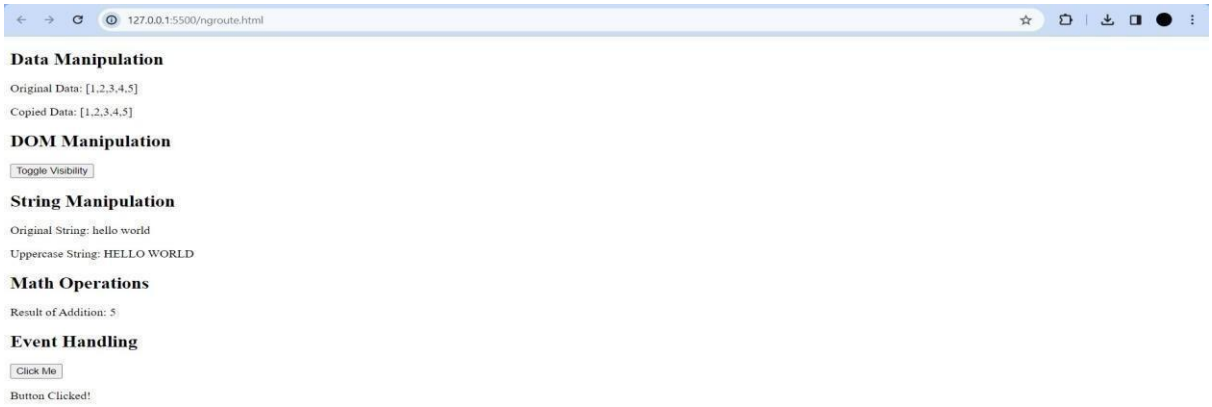
```
            $scope.toggleVisibility = function () {
                $scope.isVisible = !$scope.isVisible; };

            // String Manipulation
            $scope.originalString = "hello world";
            $scope.uppercasedString =
$scope.originalString.toUpperCase();

            // Math Operations
            $scope.additionResult = 2 + 3;

            // Event Handling
            $scope.isClicked = false;
```

```
            $scope.handleClick = function () {
                $scope.isClicked = true; };
        });
</script>

</body>
</html>
```

OUTPUT:







CONCLUSION:

In this assignment, we explored and implemented five built-in AngularJS helper functions to perform tasks such as data manipulation, DOM manipulation, and other utility operations. These functions significantly simplified complex logic, enhanced

code readability, and improved development efficiency. Overall, this practical implementation highlighted the power and versatility of AngularJS's built-in methods in building dynamic and responsive web applications.

Aarya Arban

T11 – 05

Assignment No. 9

Queries:

- Sort by year (ascending) and find the oldest car
- Make of the car that is black and built in 1977.
- Count of Ford cars
- Count of cars costing less than Rs. 100000
- Update color to red where brand is "jaguar"

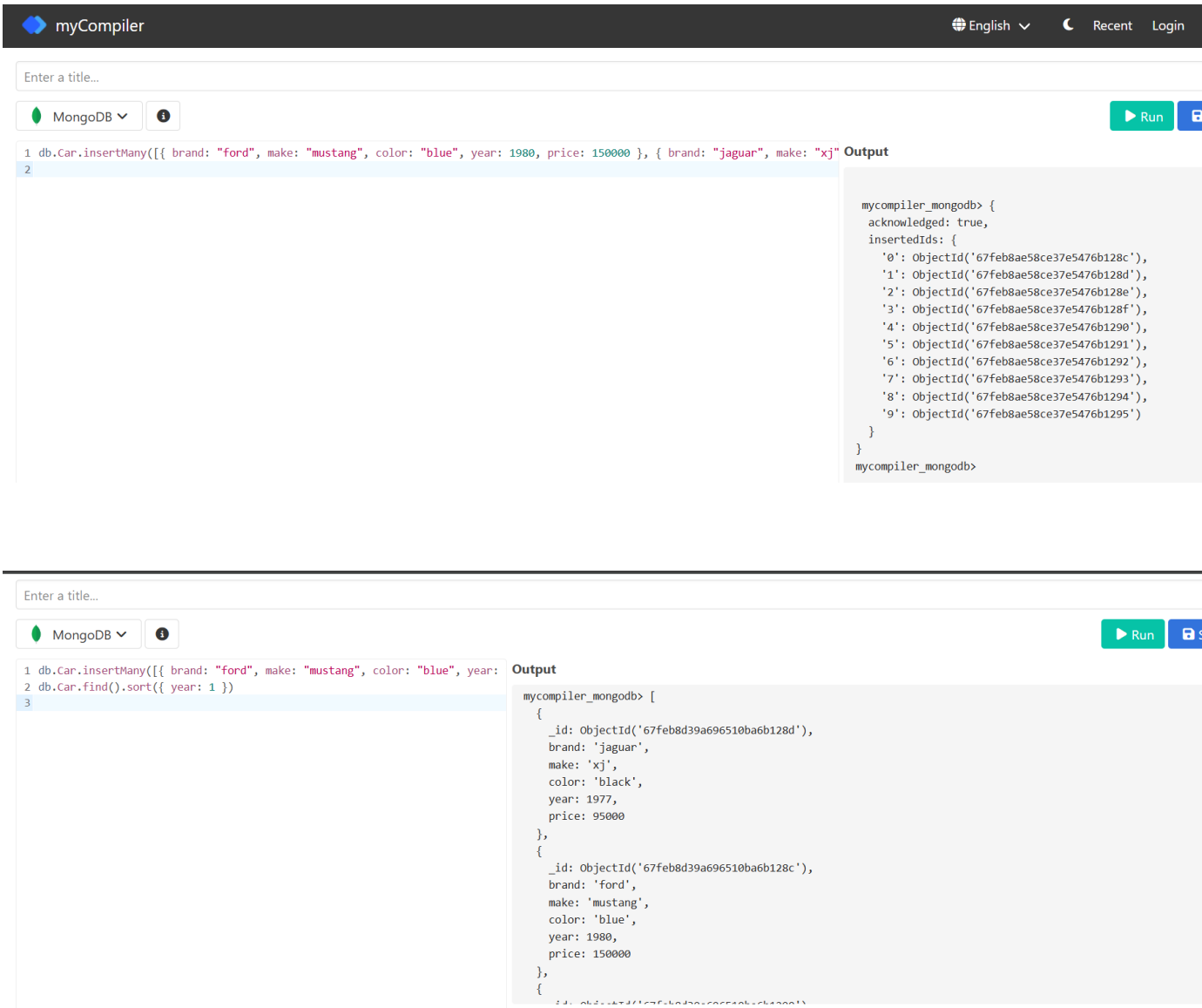
LO5: Create REST Web services using MongoDB

THEORY:

In MongoDB's command line interface (CLI), users interact directly with the database using simple commands. Starting with the `mongo` command, users connect to MongoDB and select a specific database using `use <database_name>`. Querying data involves the straightforward `db.<collection_name>.find()` command, while updating documents is achieved through `db.<collection_name>.update()`. For more complex operations, such as aggregation, users employ `db.<collection_name>.aggregate()`. Miscellaneous tasks, like deleting documents or counting entries, are handled with appropriate commands. Once tasks are completed, users exit the MongoDB CLI by typing `exit`. This direct approach allows for efficient management and manipulation of data without the need for additional tools or interfaces.

SCREENSHOTS:

- Sort by year (ascending) and find the oldest car



```
{
  _id: ObjectId('67feb8d39a696510ba6b1290'),
  brand: 'toyota',
  make: 'corolla',
  color: 'black',
  year: 2005,
  price: 90000
},
{
  _id: ObjectId('67feb8d39a696510ba6b1291'),
  brand: 'maruti',
  make: 'alto',
  color: 'white',
  year: 2008,
  price: 60000
},
{
  _id: ObjectId('67feb8d39a696510ba6b128f'),
```

```
{
  _id: ObjectId('67feb8d39a696510ba6b128f'),
  brand: 'honda',
  make: 'civic',
  color: 'red',
  year: 2010,
  price: 85000
},
{
  _id: ObjectId('67feb8d39a696510ba6b128e'),
  brand: 'ford',
  make: 'figo',
  color: 'white',
  year: 2012,
  price: 50000
},
{
  _id: ObjectId('67feb8d39a696510ba6b1295'),
```

```
  {
    _id: ObjectId('67feb8d39a696510ba6b1295'),
    brand: 'audi',
    make: 'a4',
    color: 'blue',
    year: 2015,
    price: 400000
  },
  {
    _id: ObjectId('67feb8d39a696510ba6b1292'),
    brand: 'ford',
    make: 'ecosport',
    color: 'gray',
    year: 2016,
    price: 300000
  },
  {
    _id: ObjectId('67feb8d39a696510ba6b1294'),
    brand: 'bmw',
```

```
  {
    _id: ObjectId('67feba04c23923ffee6b1294'),
    brand: 'bmw',
    make: 'x5',
    color: 'black',
    year: 2018,
    price: 550000
  },
  {
    _id: ObjectId('67feba04c23923ffee6b1293'),
    brand: 'jaguar',
    make: 'f-type',
    color: 'green',
    year: 2020,
    price: 600000
  }
]
mycompiler_mongodb>
```

- Make of the car that is black and built in 1977.

MongoDB

```
db.Car.insertMany([{ brand: "ford", make: "mustang", color: "blue", year:
db.Car.find({ color: "black", year: 1977 }, { make: 1, _id: 0 })
```

Output

```
mycompiler_mongodb> {
 acknowledged: true,
 insertedIds: {
   '0': ObjectId('67feba5cda064294b86b128c'),
   '1': ObjectId('67feba5cda064294b86b128d'),
   '2': ObjectId('67feba5cda064294b86b128e'),
   '3': ObjectId('67feba5cda064294b86b128f'),
   '4': ObjectId('67feba5cda064294b86b1290'),
   '5': ObjectId('67feba5cda064294b86b1291'),
   '6': ObjectId('67feba5cda064294b86b1292'),
   '7': ObjectId('67feba5cda064294b86b1293'),
   '8': ObjectId('67feba5cda064294b86b1294'),
   '9': ObjectId('67feba5cda064294b86b1295')
 }
}
mycompiler_mongodb> [ { make: 'xj' } ]
mycompiler_mongodb>
```

Count of Ford cars

MongoDB

```
db.Car.insertMany([{ brand: "ford", make: "mustang", color: "blue", year:
db.Car.countDocuments({ brand: "ford" })
```

Output

```
mycompiler_mongodb> {
 acknowledged: true,
 insertedIds: {
   '0': ObjectId('67febad039d9154ba36b128c'),
   '1': ObjectId('67febad039d9154ba36b128d'),
   '2': ObjectId('67febad039d9154ba36b128e'),
   '3': ObjectId('67febad039d9154ba36b128f'),
   '4': ObjectId('67febad039d9154ba36b1290'),
   '5': ObjectId('67febad039d9154ba36b1291'),
   '6': ObjectId('67febad039d9154ba36b1292'),
   '7': ObjectId('67febad039d9154ba36b1293'),
   '8': ObjectId('67febad039d9154ba36b1294'),
   '9': ObjectId('67febad039d9154ba36b1295')
 }
}
mycompiler_mongodb> 3
mycompiler_mongodb>
```

Count of cars costing less than Rs. 100000

MongoDB

```
db.Car.insertMany([{ brand: "ford", make: "mustang", color: "blue", year:
db.Car.countDocuments({ price: { $lt: 100000 } })
```

Output

```
mycompiler_mongodb> {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67febaf7abd588ea926b128c'),
    '1': ObjectId('67febaf7abd588ea926b128d'),
    '2': ObjectId('67febaf7abd588ea926b128e'),
    '3': ObjectId('67febaf7abd588ea926b128f'),
    '4': ObjectId('67febaf7abd588ea926b1290'),
    '5': ObjectId('67febaf7abd588ea926b1291'),
    '6': ObjectId('67febaf7abd588ea926b1292'),
    '7': ObjectId('67febaf7abd588ea926b1293'),
    '8': ObjectId('67febaf7abd588ea926b1294'),
    '9': ObjectId('67febaf7abd588ea926b1295')
  }
}
mycompiler_mongodb> 5
mycompiler_mongodb>
```

- Update colour to red where brand is "jaguar"

```
db.Car.insertMany([{ brand: "ford", make: "mustang", color: "blue", year:
db.Car.updateMany(
  { brand: "jaguar" },
  { $set: { color: "red" } }
)
```

Output

```
    '0': ObjectId('67febb2855d23411e16b128c'),
    '1': ObjectId('67febb2855d23411e16b128d'),
    '2': ObjectId('67febb2855d23411e16b128e'),
    '3': ObjectId('67febb2855d23411e16b128f'),
    '4': ObjectId('67febb2855d23411e16b1290'),
    '5': ObjectId('67febb2855d23411e16b1291'),
    '6': ObjectId('67febb2855d23411e16b1292'),
    '7': ObjectId('67febb2855d23411e16b1293'),
    '8': ObjectId('67febb2855d23411e16b1294'),
    '9': ObjectId('67febb2855d23411e16b1295')
  }
}
mycompiler_mongodb> ... ... ... {
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
```

CONCLUSION:

In this assignment, we successfully created a car database containing key attributes such as Brand, Make, Colour, Year, and Price. The collection was populated with at least ten documents, representing a variety of car models and manufacturers. This exercise enhanced our understanding of designing a structured and meaningful database schema, inserting multiple records efficiently, and preparing the dataset for potential use in vehicle inventory systems or automotive applications.

Aarya Arban
T11 – 05
Assignment No. 10
Aim: Design a feedback form using Flask.
LO Mapping: LO6: Design Web Applications using flask

Theory:
Flask is a lightweight and flexible **Python web framework** used for building web applications. It follows a **micro-framework** approach, meaning it provides the essential tools needed to build a web app without enforcing a strict structure or including unnecessary libraries by default. Flask is based on **Werkzeug** (a WSGI toolkit) and **Jinja2** (a templating engine). This makes it easy for beginners to start building web apps while still being powerful enough for more complex projects. Flask allows you to define routes, handle requests and responses, interact with databases, and render HTML templates, making it suitable for building both small and large-scale web applications.

Designing a web application in Flask typically involves setting up routes using Python functions, which are mapped to URLs. These functions can return HTML pages rendered with dynamic data using Jinja templates. Flask also supports form handling, session management, authentication, and database connectivity using extensions like **Flask-WTF**, **Flask-Login**, and **Flask-SQLAlchemy**. Developers can easily separate logic, templates, and static files (CSS, JS, images), which leads to clean and maintainable code. Flask's simplicity and flexibility make it a great choice for developing web apps like dashboards, blogs, portfolio sites, and even RESTful APIs.

Program:

```
<!-- APP.PY --> from flask import Flask, render_template, redirect, url_for

from flask_sqlalchemy import SQLAlchemy from flask_wtf import FlaskForm

from wtforms import StringField, TextAreaField, SelectField,

SubmitField from wtforms.validators import DataRequired, Email   app =

Flask(__name__) app.config['SECRET_KEY'] =

'your_secret_key' app.config['SQLALCHEMY_DATABASE_URI'] =

'sqlite:///feedback.db' db = SQLAlchemy(app)
```

```
# Database Model class Feedback(db.Model):     id = db.Column(db.Integer,
primary_key=True)       name = db.Column(db.String(100), nullable=False)
email = db.Column(db.String(100), nullable=False)         service_rating =
db.Column(db.String(10),    nullable=False)                  usability_rating    =
db.Column(db.String(10), nullable=False)    comments = db.Column(db.Text,
nullable=True)


# Form using Flask-WTF class FeedbackForm(FlaskForm):     name =
StringField('Name', validators=[DataRequired()])    email =
StringField('Email', validators=[DataRequired(), Email()])    service_rating =
SelectField(        'Service Rating',       choices=[
        ('1', '1 - Poor'),
        ('2', '2 - Fair'),
        ('3', '3 - Good'),
        ('4', '4 - Very Good'),
        ('5', '5 - Excellent')
    ],       validators=[DataRequired()]
  )    usability_rating =
SelectField(
```

```
        'Usability Rating',        choices=[
            ('1', '1 - Difficult'),
            ('2', '2 - Fair'),
            ('3', '3 - Good'),
            ('4', '4 - Easy'),
            ('5', '5 - Very Easy')
        ],        validators=[DataRequired()]
    )    comments = TextAreaField('Additional
Comments')    submit = SubmitField('Submit')

@app.route('/', methods=['GET', 'POST']) def feedback():
    form = FeedbackForm()    if form.validate_on_submit():        new_feedback
= Feedback(            name=form.name.data,            email=form.email.data,
service_rating=form.service_rating.data,
usability_rating=form.usability_rating.data,
comments=form.comments.data
        )
db.session.add(new_feedback)
```

```
      db.session.commit()        return redirect(url_for('thankYou'))     return
render_template('feedbackForm.html', form=form)
@app.route('/thankYou') def thank_you():
   return render_template('thankYou.html')

@app.route('/feedbackList') def feedback_list():
   feedbacks = Feedback.query.all()     return
render_template('feedbackList.html', feedbacks=feedbacks)   if __name__ ==
'__main__':     with app.app_context():
      db.create_all()     app.run(debug=True)

<!-- THANK_YOU.HTML: -->
<!DOCTYPE html>
```

```
<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Thank You</title>
```

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.m
in.css">
</head>
<body class="bg-light d-flex justify-content-center align-items-center vh-100">
<div class="card shadow p-4 text-center" style="width: 50%;">
<h2>Thank You!</h2>
<p>Your feedback has been submitted successfully.</p>
<a href="/" class="btn btn-success">Submit Another Feedback</a>
<a href="/feedback-list" class="btn btn-info">View Feedbacks</a>
</div>
</body>
</html>
<!-- FEEDBACK_LIST.HTML: -->
<!DOCTYPE html>
<html lang="en">
```

```
<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Feedback List</title> <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.m
in.css">

</head>

<body class="bg-light">
```

```
<div class="container mt-5">

<div class="card shadow p-4">

<h2 class="text-center mb-4">Submitted Feedbacks</h2>

<table class="table table-striped">

<thead>

<tr>

<th>Name</th>

<th>Email</th>

<th>Service Rating</th>

<th>Usability Rating</th>

<th>Comments</th>

</tr>

</thead>

<tbody>

{% for feedback in feedbacks %}

<tr>

<td>{{ feedback.name }}</td>

<td>{{ feedback.email }}</td>

<td>{{ feedback.service_rating }}</td>
```

```
<td>{{ feedback.usability_rating }}</td>

<td>{{ feedback.comments }}</td>

</tr>

{% endfor %}

</tbody>

</table>
```

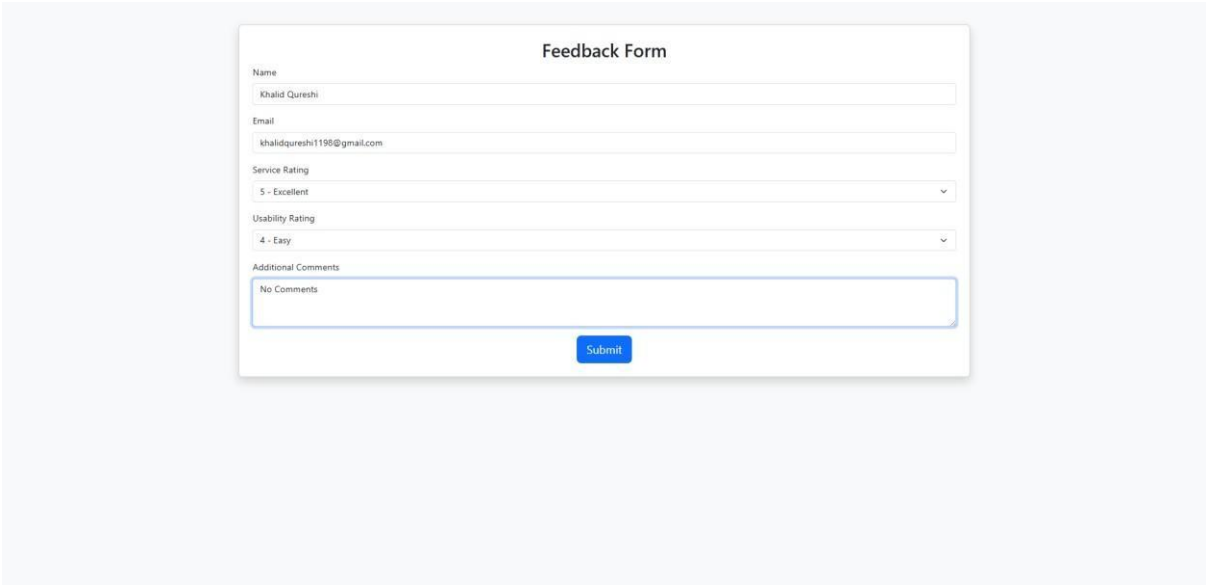
```
<a href="/" class="btn btn-primary">Submit Another Feedback</a>

</div>

</div>

</body>

</html>

<!-- FEEDBACK_FORM.HTML: -->

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Feedback Form</title> <link rel="stylesheet"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.m
in.css">

</head>

<body class="bg-light">

<div class="container mt-5">

<div class="card shadow p-4">

<h2 class="text-center">Feedback Form</h2>
```

```
<form method="POST">

{{ form.hidden_tag() }}

<div class="mb-3">

{{ form.name.label(class="form-label") }}

{{ form.name(class="form-control") }}
```

```
</div>

<div class="mb-3">

{{ form.email.label(class="form-label") }}

{{ form.email(class="form-control") }}

</div>

<div class="mb-3">

{{ form.service_rating.label(class="form-label") }}

{{ form.service_rating(class="form-select") }}

</div>

<div class="mb-3">

{{ form.usability_rating.label(class="form-label") }}

{{ form.usability_rating(class="form-select") }}

</div>

<div class="mb-3">

{{ form.comments.label(class="form-label") }}

{{ form.comments(class="form-control", rows="3") }}

</div>

<div class="text-center">

{{ form.submit(class="btn btn-primary btn-lg") }}
```

```
</div>

</form>

</div>

</div>

</body>

</html>
```

Output:


Feedback Form
Name
Khalid Qureshi
Email
khalidqureshi1198@gmail.com
Service Rating
5 - Excellent
Usability Rating
4 - Easy
Additional Comments
No Comments
Submit

Aarya Arban
T11 – 05
Assignment No. 11

Aim:
To write a program using AJAX for user validation and to show the result on the same page below the submit button.
LO Mapping: LO4: Develop Rich Internet applications using AJAX

Theory:
What is AJAX?

AJAX is a technique used in web development to create interactive and dynamic web applications. It stands for Asynchronous JavaScript and XML, although XML is not strictly required and other data formats such as JSON (JavaScript Object Notation) are commonly used.

Principles of AJAX:

1. **Asynchronous Communication:**

- AJAX allows web pages to make asynchronous requests to the server without needing to reload the entire page.
- Asynchronous requests can be made in the background while the user interacts with the page, providing a smoother user experience.

2. **Client-Side Scripting:**

- AJAX relies on client-side scripting languages such as JavaScript to handle asynchronous requests and update the page content dynamically.

3. **Server-Side Processing:**

● AJAX requests are typically handled by server-side scripts or APIs that process the request, perform any necessary operations (such as database queries or calculations), and return a response.

4. **Data Exchange Formats:**

● AJAX requests can exchange data with the server in various formats, including XML, JSON, plain text, or HTML.

● JSON (JavaScript Object Notation) is commonly used due to its lightweight and easy-to-parse nature.

5. **DOM Manipulation:**

- Once the response is received from the server, the client-side script can dynamically update the DOM (Document Object Model) to reflect the changes without requiring a page reload.
- This enables creating dynamic user interfaces and real-time updates.

Usage of AJAX:

1. **Form Submission:**

- AJAX is often used to submit form data to the server without reloading the entire page.
- This allows for a more seamless user experience and faster response times.

2. **Data Retrieval:**

- AJAX can retrieve data from the server in response to user actions, such as clicking a button or scrolling a page.
- This data can be used to dynamically update the page content, such as fetching new articles in a news feed or loading additional product listings in an e-commerce website.

3. **Real-Time Updates:**

- AJAX enables real-time updates to web applications by periodically fetching new data from the server and updating the page content accordingly.
- This is commonly used in chat applications, social media feeds, and live sports scores.

4. **Autosave Functionality:**

● AJAX can be used to implement autosave functionality in web forms, where form data is periodically sent to the server in the background to prevent data loss in case of browser crashes or accidental navigation away from the page.

5. **Interactive Web Applications:**

● AJAX is essential for creating interactive and responsive web applications, where user actions trigger asynchronous requests to the server, and the page content is updated dynamically without full page reloads.

Advantages of AJAX:

1. **Improved User Experience:**

● AJAX enables smoother and more responsive web applications by reducing page reloads and providing real-time updates.

2. **Reduced Server Load:**

● Asynchronous requests allow for selective updates of page content, reducing the amount of data transferred between the client and server and lowering server load.

3. **Faster Performance:**

● By fetching data asynchronously, AJAX requests can improve performance by fetching only the necessary data and updating the page dynamically, without waiting for the entire page to reload.

4. **Enhanced Interactivity:**

● AJAX enables rich and interactive user interfaces by allowing for seamless updates to page content based on user actions.

5. **Cross-Browser Compatibility:**

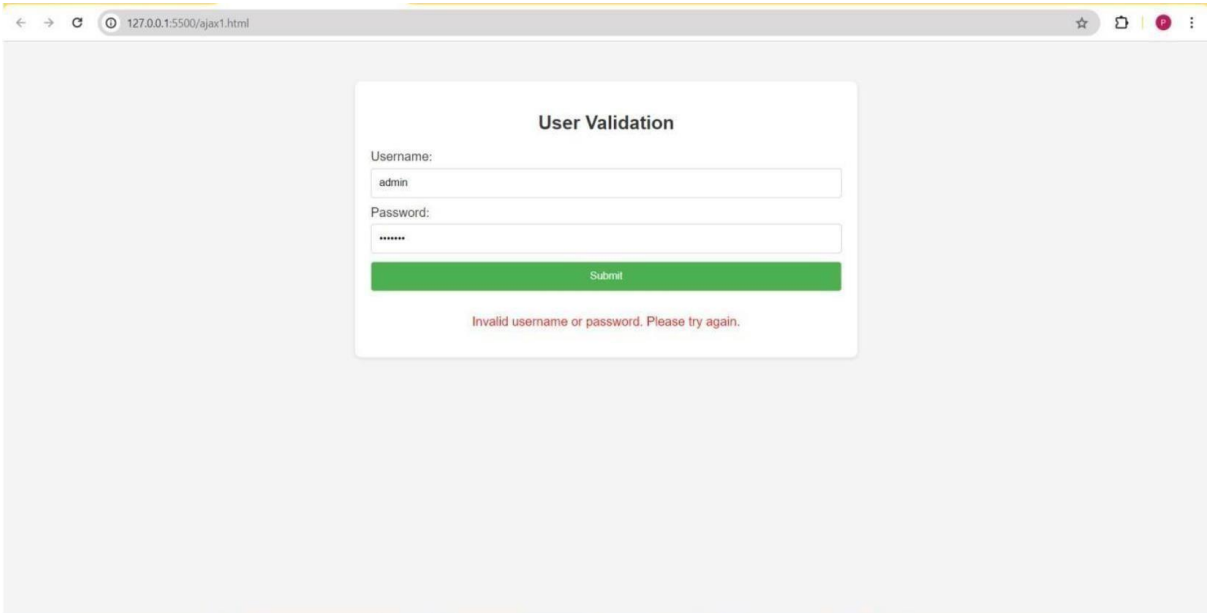
- AJAX is supported by all major web browsers, making it a reliable and widely adopted technique for web development.

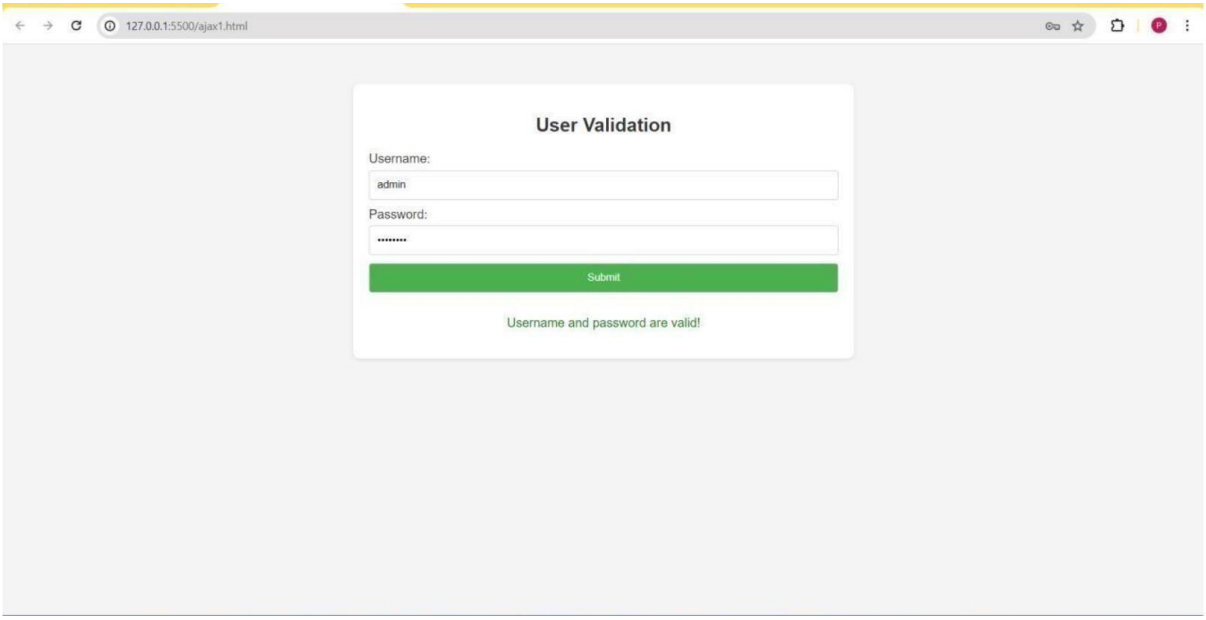
Code:

```
<body>
<div class="container">
  <h2>User Validation</h2>
  <form id="validationForm">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required><br>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required><br>
    <button type="submit" id="submitBtn">Submit</button>
  </form>

  <div id="validationResult"></div>
</div>

<script>
document.getElementById('validationForm').addEventListener('submit', function(event){
    event.preventDefault(); // Prevent form submission

    var username = document.getElementById('username').value;
    var password = document.getElementById('password').value;

    // Basic client-side validation
    if(username.trim() === "" || password.trim() === "") {
        document.getElementById('validationResult').innerHTML = "<p id='errorMsg'>Please enter both username and password.</p>";
        return;
    }

    // AJAX request to simulate server-side validation
    var xhr = new XMLHttpRequest();
    xhr.open('GET', 'response.json', true); // Point to a JSON file containing response
    xhr.setRequestHeader('Content-Type', 'application/json');
```

```
<script>
document.getElementById('validationForm').addEventListener('submit', function(event){
    // AJAX request to simulate server-side validation
    var xhr = new XMLHttpRequest();
    xhr.open('GET', 'response.json', true); // Point to a JSON file containing response
    xhr.setRequestHeader('Content-Type', 'application/json');

    xhr.onload = function() {
        if (xhr.status === 200) {
            var response = JSON.parse(xhr.responseText);
            if (response.validCredentials.hasOwnProperty(username) && response.validCredentials[username] === password) {
                document.getElementById('validationResult').innerHTML = "<p id='successMsg'>Username and password are valid!</p>";
            } else {
                document.getElementById('validationResult').innerHTML = "<p id='errorMsg'>Invalid username or password. Please try again.</p>";
            }
        } else {
            console.log('Request failed. Status: ' + xhr.status);
        }
    };
    xhr.send();
});
</script>

</body>
</html>
```

Output:





Conclusion:

AJAX is a powerful technique that revolutionized web development by enabling the creation of interactive, dynamic, and responsive web applications. By leveraging asynchronous communication between the client and server, AJAX provides a smoother user experience, faster performance,

and enhanced interactivity, making it an essential tool for modern web development.